Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (Raise Hand).
When instructor calls on you, click microphone icon to unmute. When you've finished speaking, *be sure to mute yourself again*.

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class for their own private use. Releasing these recordings is strictly prohibited.

THE GEORGE WASHINGTON UNIVERSITY
WASHINGTON, DC

SEAS 8515

Data Engineering for AI

Professor Steven Cocke

Week 3: May 30, 2026

Agenda

- HonorLock Practice Exam
- Week 2 Assignment Review
- Introduction to Data Engineering
  - Data Engineering Lifecycle
  - Data Engineering Undercurrents
- Storage & Distributed Computing Concepts
- Assignment Overview for This Week

HonorLock Practice Exam

- Available at 9am ET on 5/30 (today)
- Open until 6/27
- Not graded
- Please complete it to test out your set up and ensure there are no issues
- I am limited in my capacity to assist during an exam to help with technical issues, so please test your system early!

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Week 2 Homework Solution

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Introduction to Data Engineering

ML in production: Expectation

Expectation: In an ideal world, deploying machine learning models would be a straightforward process:

Step 1: Collect Data - Gather a relevant dataset.
Step 2: Train Model - Apply ML algorithms to train the model.
Step 3: Deploy Model - Move the model to production and watch it perform optimally.
Expected Outcome: The model works seamlessly, generating valuable insights or revenue with minimal issues.

ML in production: Reality

Deploying machine learning models is rarely straightforward. It's an iterative process involving continuous adjustments, data handling, and feedback loops. Here's a look at what often happens in practice:

Choosing a Metric to Optimize
Success begins with choosing relevant performance metrics (accuracy, F1 score, precision/recall, etc.) that align with project objectives. However, selecting the wrong metric can lead to misleading results.

Data Collection
Gathering data for training is a continuous process. In production, new data must be collected, processed, and integrated regularly to maintain model accuracy.

Training and Retraining the Model
Training often reveals data quality issues, requiring relabeling, cleaning, and, sometimes, starting over. Each iteration aims to improve the model, but new issues can arise, such as poor performance on specific classes or overfitting.

ML in production: Reality

Handling Edge Cases and Biases
After deployment, user feedback and new data may expose model biases or poor performance on edge cases. This often necessitates a rollback to previous versions or extensive retraining.

Scaling and Monitoring
Once deployed, scaling the model to handle larger workloads can reveal new bottlenecks. Continuous monitoring is essential to detect drifts in model accuracy, requiring frequent updates and, occasionally, a complete retraining pipeline.

Dealing with Setbacks
Deployment rarely goes as planned. There may be instances where performance drops, revenue impacts, or operational issues force teams to revisit earlier steps, adjust metrics, or even start from scratch.

Iterate and Improve
The cycle of collecting new data, retraining, deploying, and monitoring is ongoing. Each iteration aims to refine the model, but perfection is challenging in dynamic, real-world environments..

Project considerations

1. Framing

- Effective project framing sets the foundation. This project addresses the challenges of deploying machine learning models in real-world environments, with a focus on optimizing performance, handling data challenges, and ensuring operational stability.
- Key Question: Why is this project essential? The goal is to identify and mitigate issues that often arise post-deployment, improving model robustness and reliability.

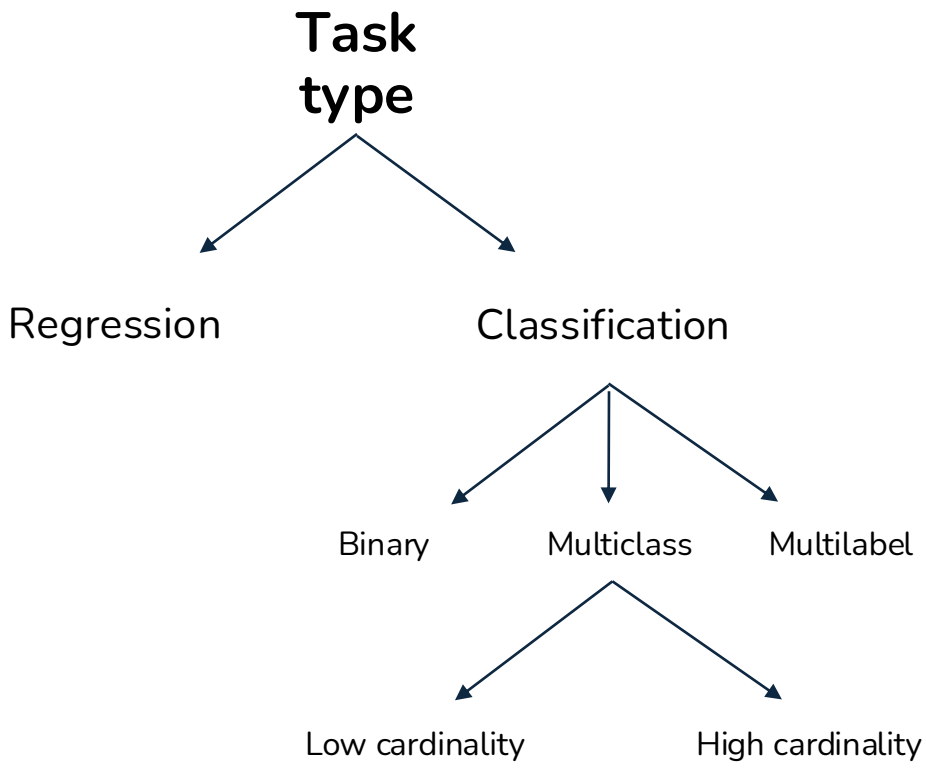
2. Objectives

Define clear, measurable objectives to guide the project. Objectives include:

- Enhancing model accuracy and stability across diverse datasets.
- Reducing the time and resources needed for retraining and model updates.
- Building a feedback loop for continuous model improvement.

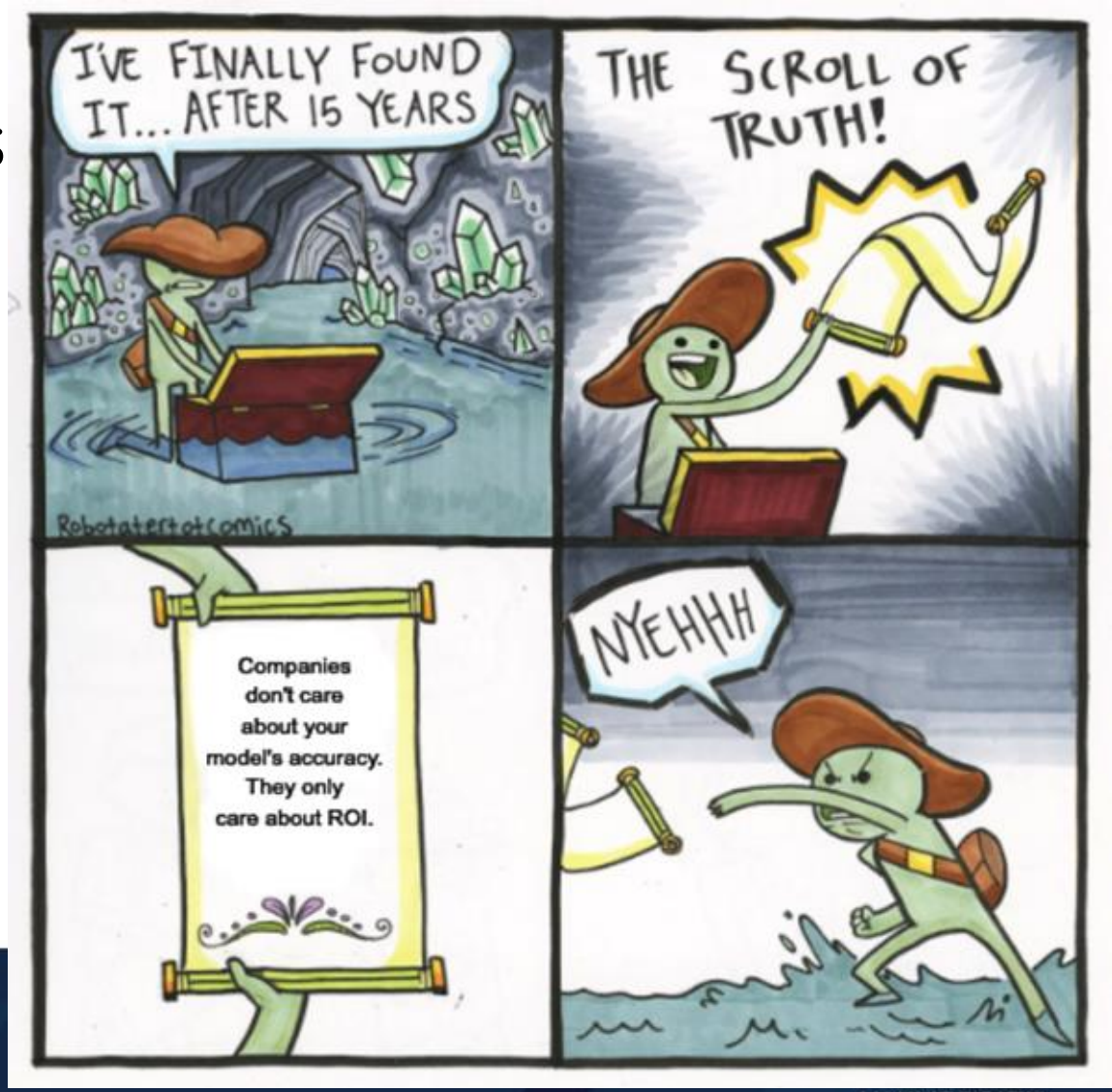
Note: Objectives should align with business needs and stakeholder expectations.

Framing the problem

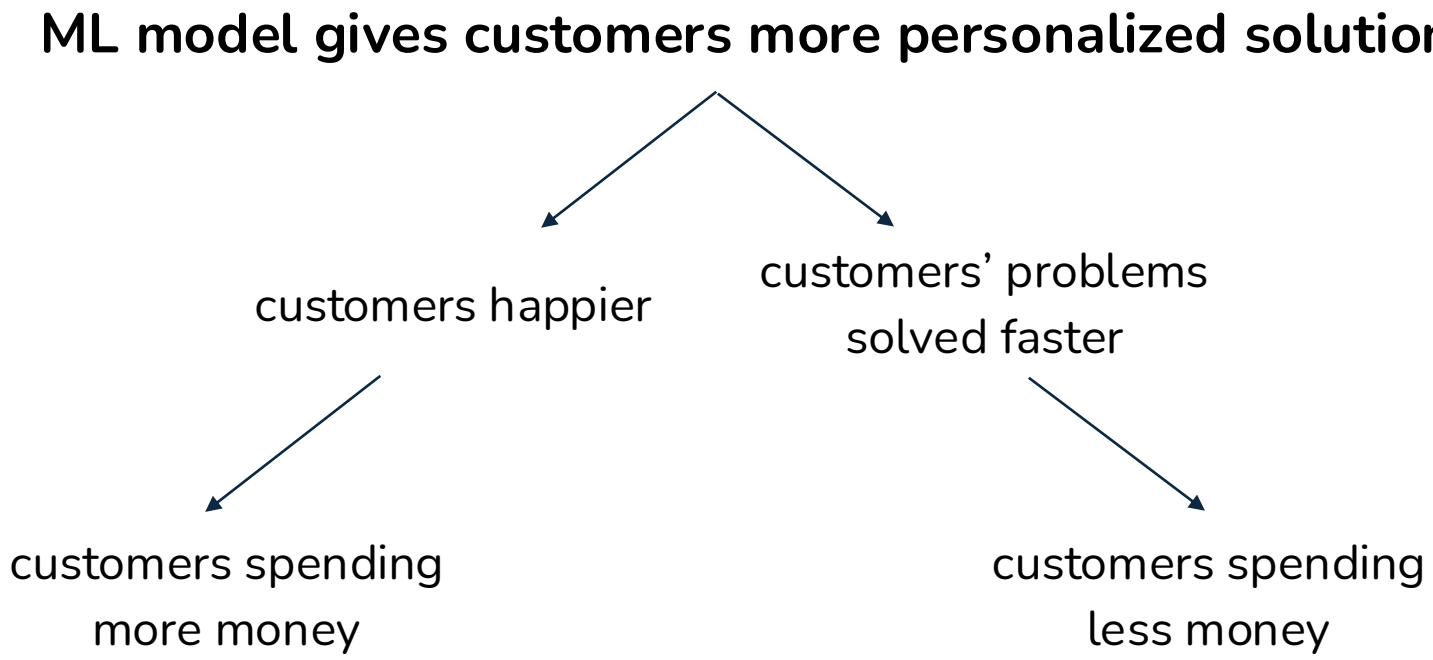


Project objectives

- ML objectives
  - Performance
  - Latency
  - etc.
- Business objectives
  - Cost
  - ROI
  - Regulation & compliance



ML <-> business: can be tricky



Project considerations

3. Constraints

Every project has limitations. Common constraints for ML deployment projects include:

- Data Constraints: Limited labeled data, data privacy concerns, and the challenge of obtaining diverse, high-quality datasets.
- Resource Constraints: Computational power, storage, and budgetary limits.
- Operational Constraints: Time limitations for deployment, user expectations, and the need for seamless integration.

Understanding these constraints helps in planning and prioritizing efforts.

4. Phases

Organizing the project into phases provides structure:

- Phase 1: Data Collection & Preparation - Gathering relevant data, preprocessing, and handling missing values.
- Phase 2: Model Development & Training - Building, training, and fine-tuning the model to meet desired performance.
- Phase 3: Evaluation & Testing - Validating the model on test data, identifying any weaknesses, and ensuring performance benchmarks are met.
- Phase 4: Deployment & Monitoring - Deploying the model into production, setting up monitoring systems, and establishing retraining schedules.

Business objectives

How can this ML project increase profits directly or indirectly?

- Directly: increasing sales (ads, conversion rates), cutting costs
- Indirectly: increasing customer satisfaction, increasing time spent on a website
- Baselines
  - Existing solutions, simple solutions, human experts, competitors solutions, etc.
- Usefulness threshold
  - Self-driving needs human-level performance. Predictive texting doesn't.
- False negatives vs. false positives
  - Covid screening: no false negative (patients with covid shouldn't be classified as no covid)
  - Fingerprint unlocking: no false positive (unauthorized people shouldn't be given access)
- Interpretability
  - Does it need to be interpretable? If yes, to whom?
- Confidence measurement (how confident it is about a prediction)
  - Does it need confidence measurement?
  - Is there a confidence threshold? What to do with predictions below that threshold—discard it, loop in humans, or ask for more information from users?

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

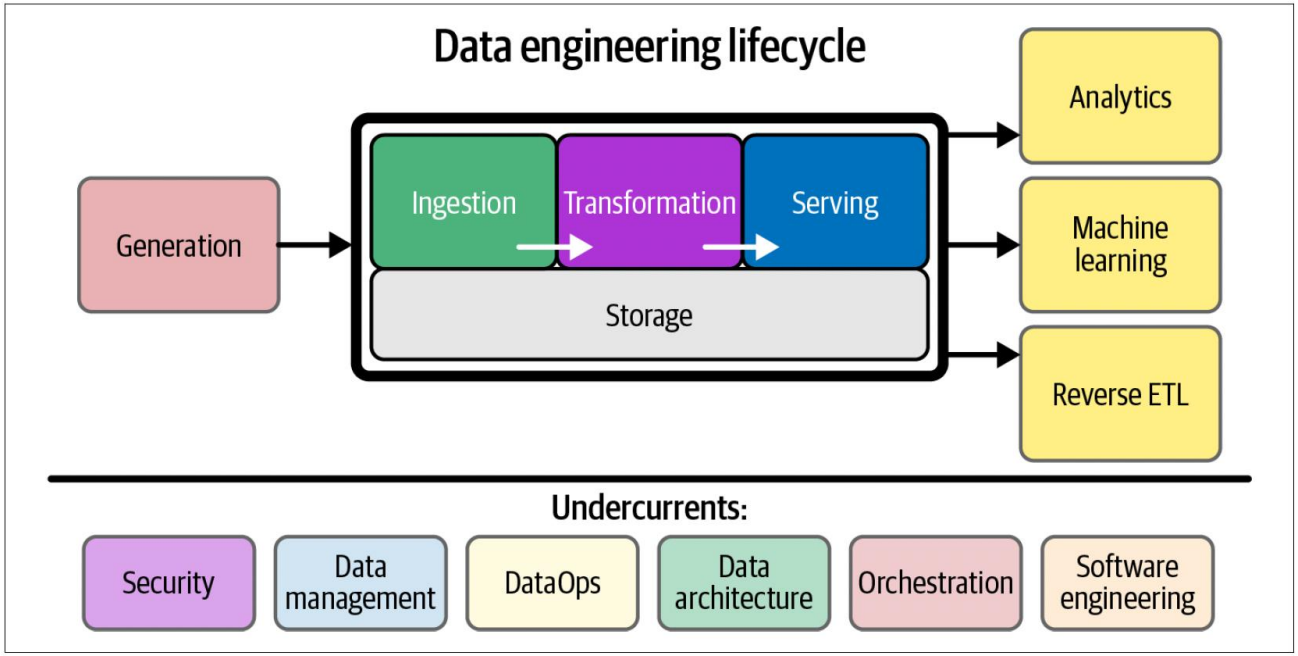
Data Engineering Lifecycle

What is the Data Engineering Lifecycle?

Definition:
The data engineering lifecycle transforms raw data into usable assets for analysts, data scientists, ML engineers, and others.

Five Key Stages:
1. Generation – Data originates from source systems.
2. Storage – Data is stored at various points and acts as the foundation.
3. Ingestion – Data is collected into processing systems.
4. Transformation – Data is cleaned, enriched, and structured.
5. Serving – Data is made available for downstream users and applications.

Note: These stages may repeat, overlap, or occur out of order in some cases.



Lifecycle Dynamics and Supporting Undercurrents

Dynamic Flow:

- Lifecycle is not always linear.
- Middle stages (Storage, Ingestion, Transformation) often intertwine.
- Flexibility is essential—real-world pipelines evolve over time.

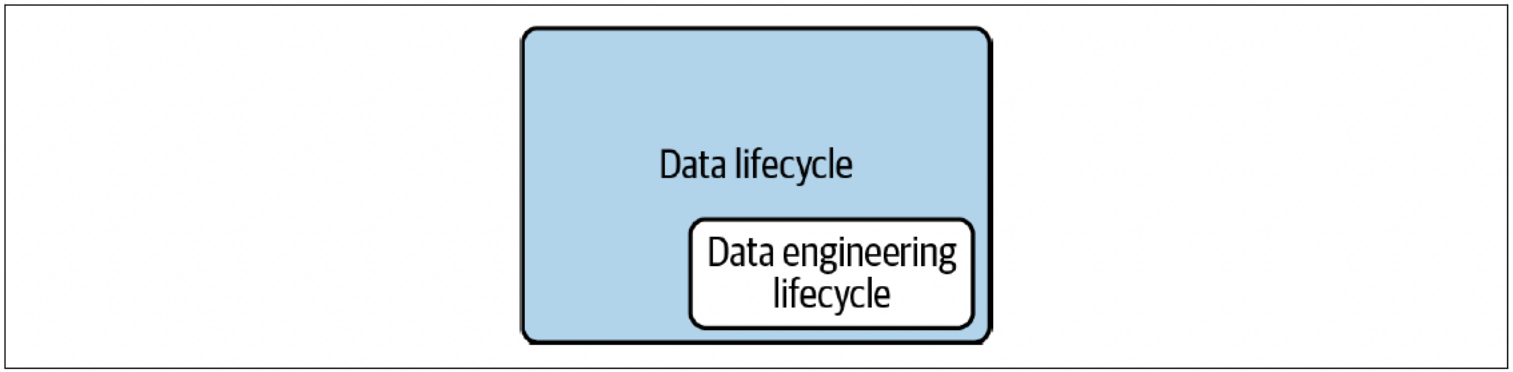
Foundational Undercurrents (span across stages):

- Security
- Data Management
- Orchestration & DataOps
- Data Architecture
- Software Engineering

These undercurrents ensure the lifecycle runs reliably, securely, and at scale.

The Data Lifecycle Versus the Data Engineering Lifecycle

The data engineering lifecycle is a subset of the broader data lifecycle. While the full data lifecycle spans the entire lifespan of data, the engineering lifecycle focuses on stages directly managed by data engineers.



Understanding Source Systems in Data Engineering

Definition:
A source system is the origin of data used in the data engineering lifecycle.

Examples:
- IoT devices
- Application message queues
- Transactional databases

Key Responsibilities for Data Engineers:
- Understand how data is generated (frequency, velocity, variety).
- Maintain communication with source owners about changes.
- Adapt pipelines to structural or tech stack changes (e.g., field updates, DB migrations).

Data engineers do not control source systems. They adapt to them.

Understanding Source Systems in Data Engineering

Two Common Examples of Source Systems

Traditional Application Database

- Originated in the 1980s with RDBMS
- Application servers connect to a central database
- Still widely used, now often with microservices and distributed databases

IoT Swarms

- Modern, high-velocity data source
- Involves streams of sensor data from numerous devices
- Presents unique scaling and processing challenges

Source

Evaluating Source Systems

When assessing a source system, data engineers need to understand the nature of the system and how it produces data. Is it an application backend, a stream of IoT signals, or a human-managed spreadsheet?

Key questions include:

- How is the data stored? Temporarily or long term?
- At what rate is data generated (events per second, size per hour)?
- How clean and consistent is the data? Are there frequent nulls, formatting issues, or out-of-order records?
- Will errors or duplicate values be common?
- Do some records arrive late or much later than others?

These insights are foundational for designing reliable pipelines.

Engineering Considerations for Integration

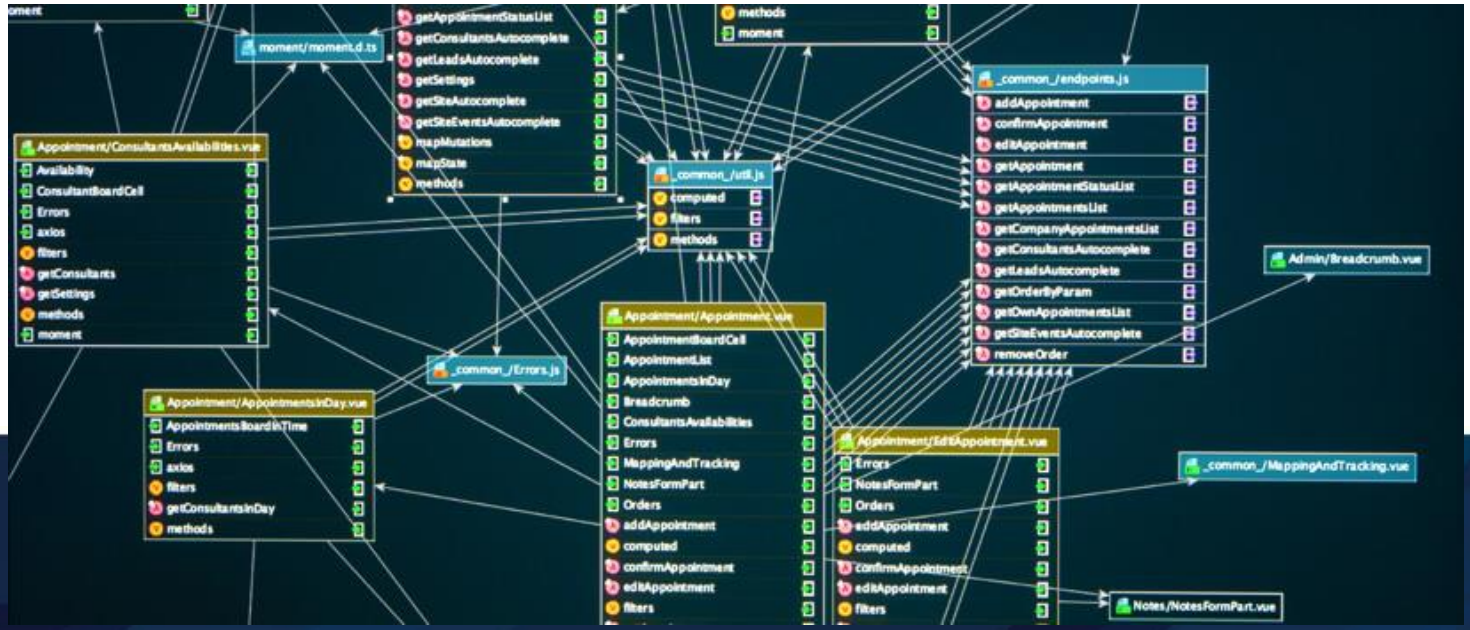
Beyond raw data, engineers must consider how the system behaves operationally.

- What is the structure of the data? Does it involve a single flat table or multiple related datasets across systems?
- How is schema evolution handled, and how are downstream teams notified when changes occur?
- How frequently should the system be queried or polled for updates?
- Does the source provide full snapshots or change events (CDC)?
- Who is responsible for providing the data, and are there any upstream dependencies that could impact quality?
- Will querying the source system for analytics affect its performance?

These factors influence everything from pipeline design to long-term maintainability.

Schema Complexity and the Role of the Data Engineer

- Understanding and managing schema is one of the most challenging aspects of working with source systems.
- Some systems use fixed schemas, like relational databases, where structure is enforced. Others are schema-less, like log streams or document stores, where the schema is inferred as data is written. Both approaches introduce complexity, especially as schemas evolve over time.
- Data engineers are responsible for transforming raw input into structured, analytics-ready outputs. This requires handling frequent changes, adapting to new fields, and ensuring that downstream systems remain stable even as the shape of the data shifts. Schema evolution isn't a rare event. It's a constant reality in modern data systems.



Source

Storage

When we talk about storing data, we're not just talking about where it *sits*.
We're talking about how it *moves*, how it's *processed*, and how it's *used*.

- **Object stores** (like Amazon S3) don't just hold files; they support SQL-like queries.
- **Streaming platforms** (like Kafka or Pulsar) store, ingest, and let you query all in one.
- **Data warehouses** double as transformation engines and analytics tools.

Storage crosses boundaries. It touches ingestion, transformation, serving, and even real-time processing. Choosing a storage solution means deciding:

- How fast your pipeline runs
- How scalable your system is
- How your users interact with the data

*Storage is a strategy (more to come).

Evaluating storage systems: Key engineering considerations

Choosing a storage layer isn't just about where data lives. It's also about how the entire system performs now and scales in the future. Here are a few questions every data engineer should ask:

- Does this storage meet required read/write speeds?
- Will it become a bottleneck for downstream systems?
- Are you using the technology as intended, or are you forcing it into an antipattern (e.g., random updates in object storage)?
- Can it handle future scale, not just volume, but throughput and concurrency?
- Will downstream consumers be able to access data within their service level agreement (SLA)?
- Are you capturing metadata for schema changes, lineage, and flow?

Good storage is more than capacity; it's about performance, usability, and trust in every stage of the data lifecycle.

Storage Capabilities, Governance, and Compliance

Not all storage systems are created equal. Understanding their capabilities and constraints is essential for building reliable, compliant, and scalable data infrastructure.

- Is this a pure storage system like object storage, or does it support complex query and processing workflows like a cloud data warehouse?
- What kind of schema behavior does it support? Is it schema-agnostic, schema-flexible, or schema-enforcing?
- How are you managing master data, golden records, and data lineage to support governance and ensure long-term data quality?
- Are regulatory compliance and data sovereignty requirements addressed? Can you control where your data physically resides?

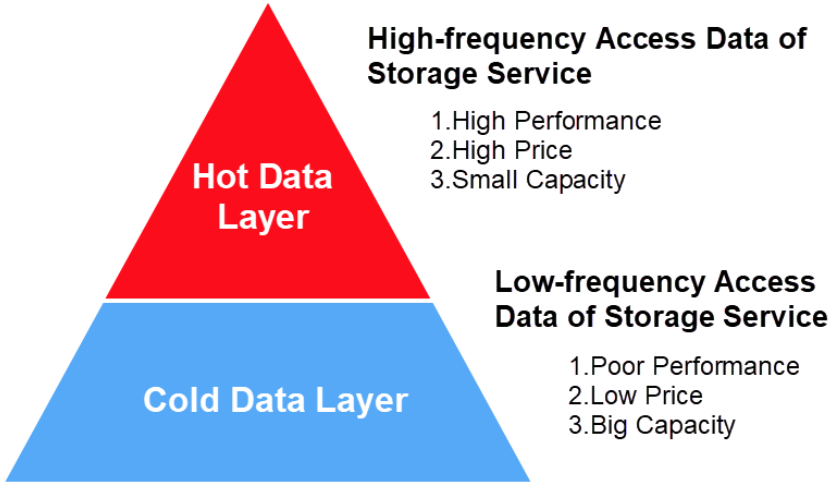
Choosing the right storage isn't only about performance; it's about governance, control, and the ability to evolve responsibly.

Understanding Data Access Frequency

Not all data is accessed equally. Retrieval patterns vary depending on the use case, which introduces the idea of data temperature.

- **Hot data** is accessed frequently...and sometimes multiple times per second. This is the data powering real-time systems and user-facing applications. It should be stored on systems optimized for fast retrieval.
- **Lukewarm data** is accessed occasionally, such as once a week or month. It doesn't need ultra-low latency, but it still must be readily accessible.
- **Cold data** is rarely queried. It's typically stored for compliance, backups, or disaster recovery. In cloud environments, cold data is often placed in low-cost archival tiers with slower and more expensive retrieval options.

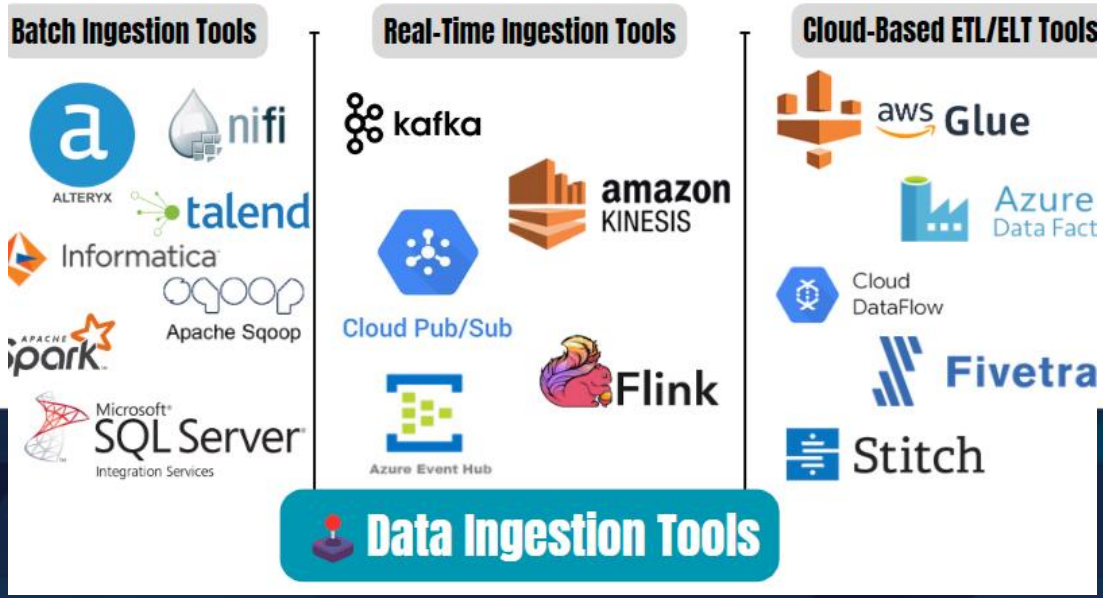
Choosing the right storage tier based on access frequency ensures both performance and cost efficiency.



Source

Ingestion – Where Data Enters the Pipeline

- Once you understand your source systems and storage layers, the next critical step is ingestion, meaning getting the data into your system.
- Ingestion is often the most fragile part of the data engineering lifecycle. Source systems are usually external and may behave unpredictably. They can go offline, deliver incomplete or low-quality data, or break downstream services without warning.
- Ingestion services themselves can also fail, halting the flow of data needed for storage, processing, and serving. A failure at this stage creates ripple effects across the entire pipeline.
- Reliable ingestion is essential. It's the bridge between upstream complexity and downstream usability.



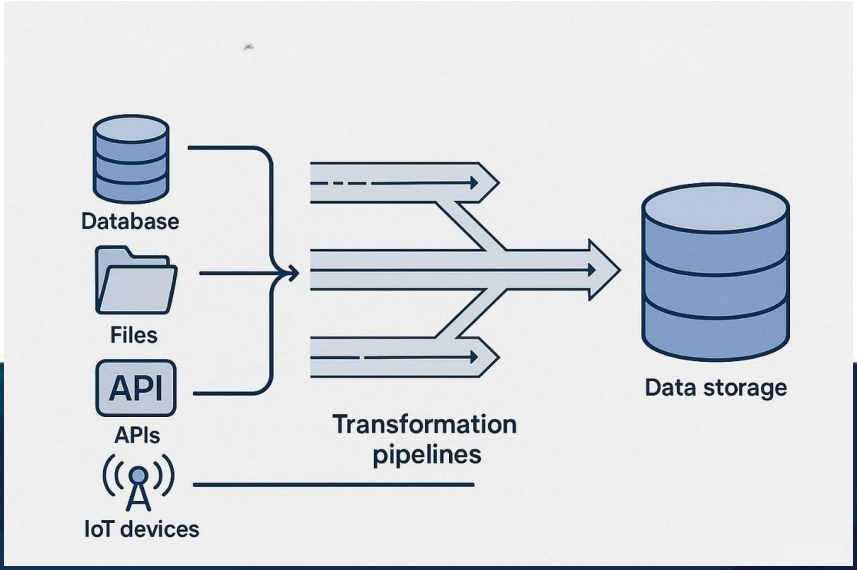
Source

Key Questions for Designing the Ingestion Layer

Before building or scaling your ingestion system, consider the following:

- What are the core use cases for this data? Can it be reused to avoid duplication?
- How reliable are the systems producing and ingesting the data?
- Where is the data going after ingestion?
- How often will the data be accessed?
- What volume should be expected, and in what format does it arrive?
- Can downstream systems handle the data format as-is?
- Is the source data usable immediately, or does it degrade over time?
- For streaming data, is any transformation required before it reaches its destination?

Careful planning at the ingestion stage ensures stable, scalable, and cost-efficient data pipelines.



Source

Batch versus Streaming

All data starts as a stream and it's produced continuously at the source. The choice between batch and streaming is about *how* we process and deliver that data downstream.

- **Batch ingestion** processes large chunks of data at fixed intervals or when a size threshold is reached. It's widely used in analytics and ML workflows and remains popular due to its simplicity and compatibility with legacy systems.
- **Streaming ingestion** delivers data continuously and with low latency, often in near real-time. Data is made available to downstream systems shortly after it is generated…sometimes within milliseconds.

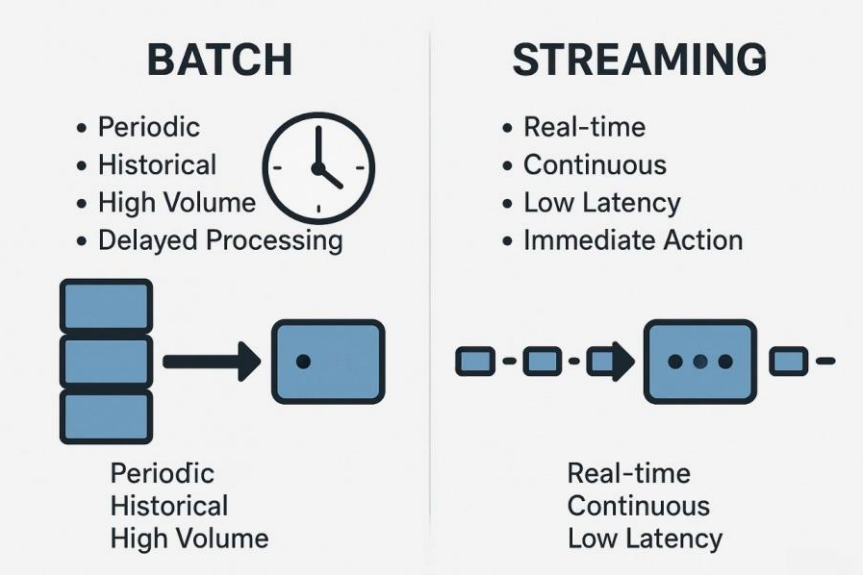
Batch is efficient for periodic analysis, but introduces inherent delay. Streaming enables timelier insights and responses, especially in operational and event-driven systems. The right choice depends on the use case, the importance of freshness, and the capabilities of the surrounding architecture.

Batch or Streaming? Ask These First

Before going streaming-first, consider:

- Can downstream systems handle real-time flow?
- Is true real-time needed, or is micro-batching enough?
- What value does streaming add over batch?
- Are the added costs and complexity justified?
- Is your pipeline resilient to failures?
- Will you use managed tools or self-hosted infrastructure?
- Does your ML model benefit from real-time predictions?
- Will ingestion impact the source system?

Batch works well for many use cases—use streaming when real-time truly matters.



Source

Push vs. Pull – Two Ingestion Patterns

In data ingestion, data can either be **pushed** from the source or **pulled** by the destination. Both patterns appear across pipelines, often in combination.

- **Push model:** The source sends data directly to a target (e.g., a queue, object store, or API endpoint). Common in real-time systems and IoT fleets.
- **Pull model:** The ingestion system fetches data from the source on a schedule. Typical in batch extract-transform-load (ETL) or timestamp-based change data capture (CDC).

Examples:

- **ETL:** Pulls a snapshot from source tables.
- **CDC with logs:** The database pushes to binary logs; the system reads them without hitting the database.
- **Streaming:** Events are pushed directly from producers, often through platforms like Kafka or Kinesis.

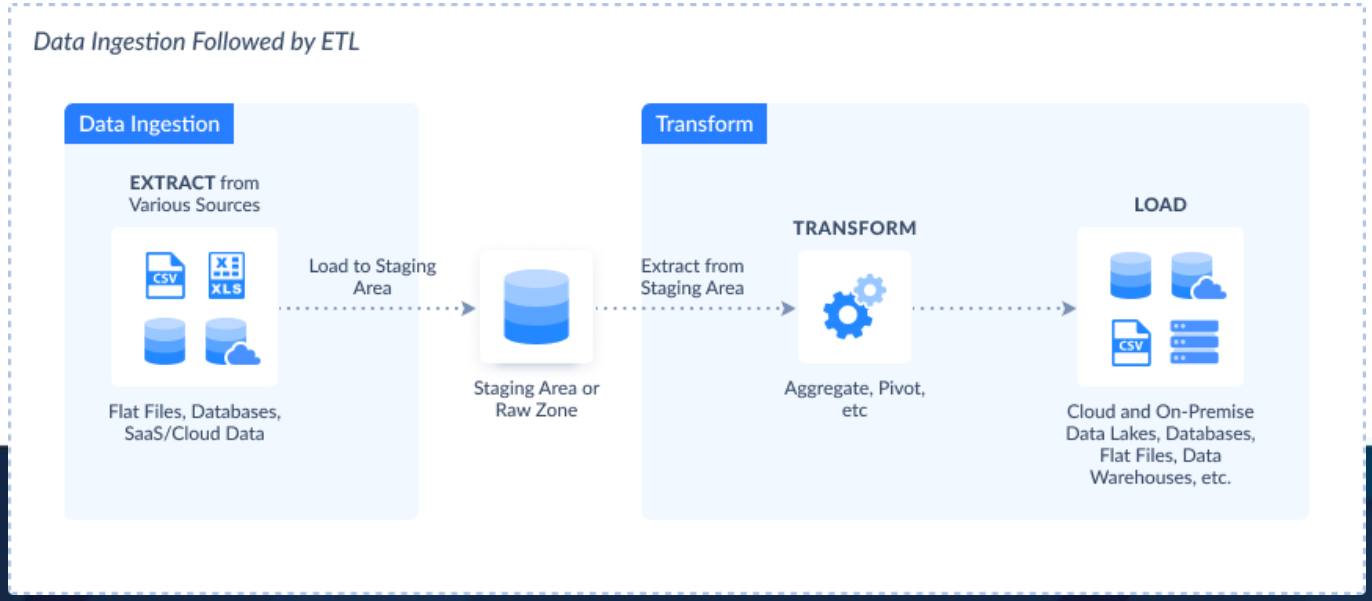
Choosing between push and pull depends on system control, latency needs, and data freshness.

Transformation – Turning Raw Data into Value

Once data is ingested and stored, it must be transformed into a usable format. This stage is where raw data becomes meaningful for analytics, reporting, and machine learning.

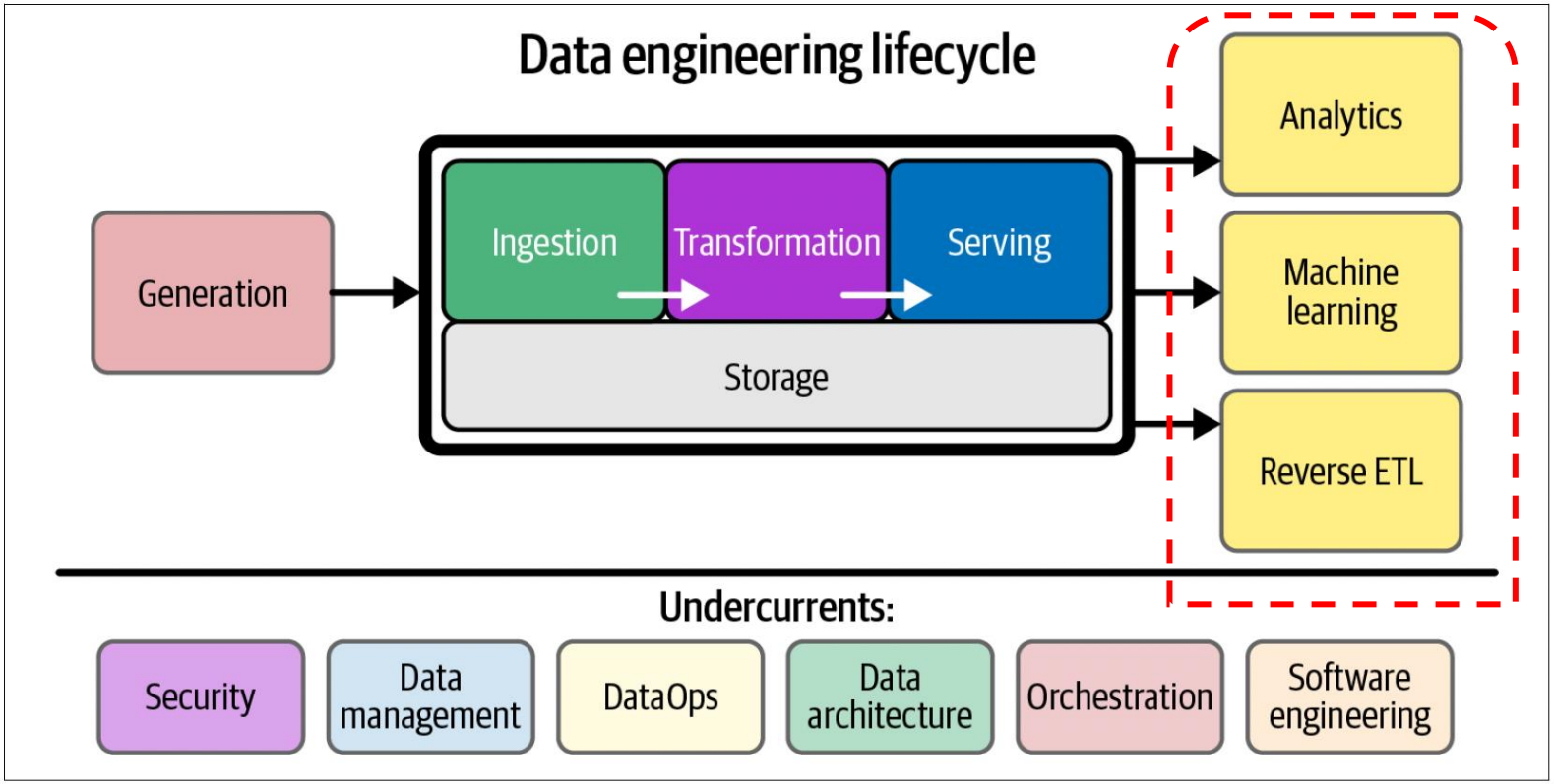
- Early transformations include type casting, standardizing formats, and filtering out bad records.
- Later steps may involve reshaping schemas, normalizing data, and aggregating at scale.
- Transformation is where data begins to deliver value...without it, data remains inert and unusable.

This stage connects raw input to real-world insight.



Source

Last Stage -> Serving Data



Serving Data

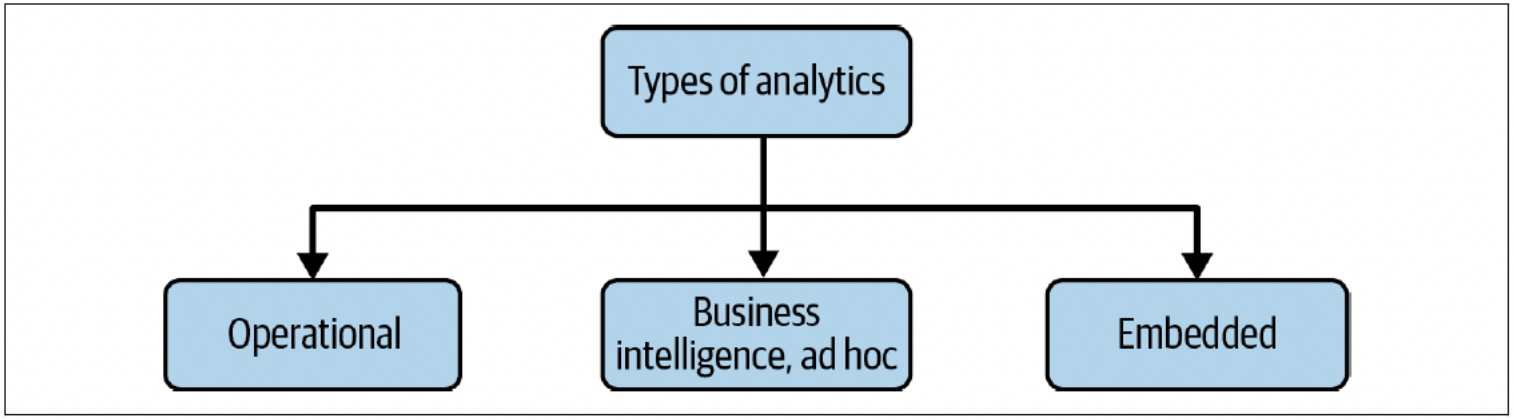
This is the final and most impactful stage of the data engineering lifecycle. Once data has been ingested, stored, and transformed, it's time to deliver it to the people and systems that will use it.

- **Data only has value when it's used.** If it's never queried, analyzed, or acted upon, it's just taking up space.
- Many organizations fall into the trap of data vanity projects—collecting massive volumes of data that serve no purpose.
- Data must be served with intention, aligned with real business needs.

Serving is where analytics happens, machine learning models are deployed, and reverse ETL powers operational tools. This is where data stops being infrastructure and starts becoming impact.

Analytics

- Once data is stored and transformed, it powers reporting, dashboards, and ad hoc analysis.
- Modern analytics goes beyond BI, now including operational and embedded analytics for real-time and in-product insights.



Variants of Analytics – BI, Operational, Embedded

Business Intelligence (BI):
Describes the past and present using structured reports and dashboards. BI relies on clear business logic. This is applied either during transformation or at query time (logic-on-read). Self-service analytics is the goal, but data quality and organizational barriers often stand in the way.

Operational & Embedded Analytics:
- **Operational analytics** delivers real-time, actionable insights (e.g., inventory views, live app health).
- **Embedded analytics** brings data to customers inside products, requiring strict access controls and scalable performance. Data leaks here are critical risks, not just internal mistakes.

Machine Learning

ML becomes viable when a company reaches strong data maturity. But success in ML depends on the strength of your data engineering.

- Data engineers support ML by managing pipelines, feature stores, metadata, and orchestration tools.
- Collaboration between data, ML, and analytics teams is critical. Clear boundaries and shared tools are key to moving fast without chaos.
- Feature stores help scale ML by tracking feature versions, enabling reuse, and reducing operational overhead.
- Before investing in ML, ensure your data is high quality, discoverable, unbiased, and well-modeled.

ML is exciting, but don't skip the groundwork. Master analytics first and then build up.

Reverse ETL

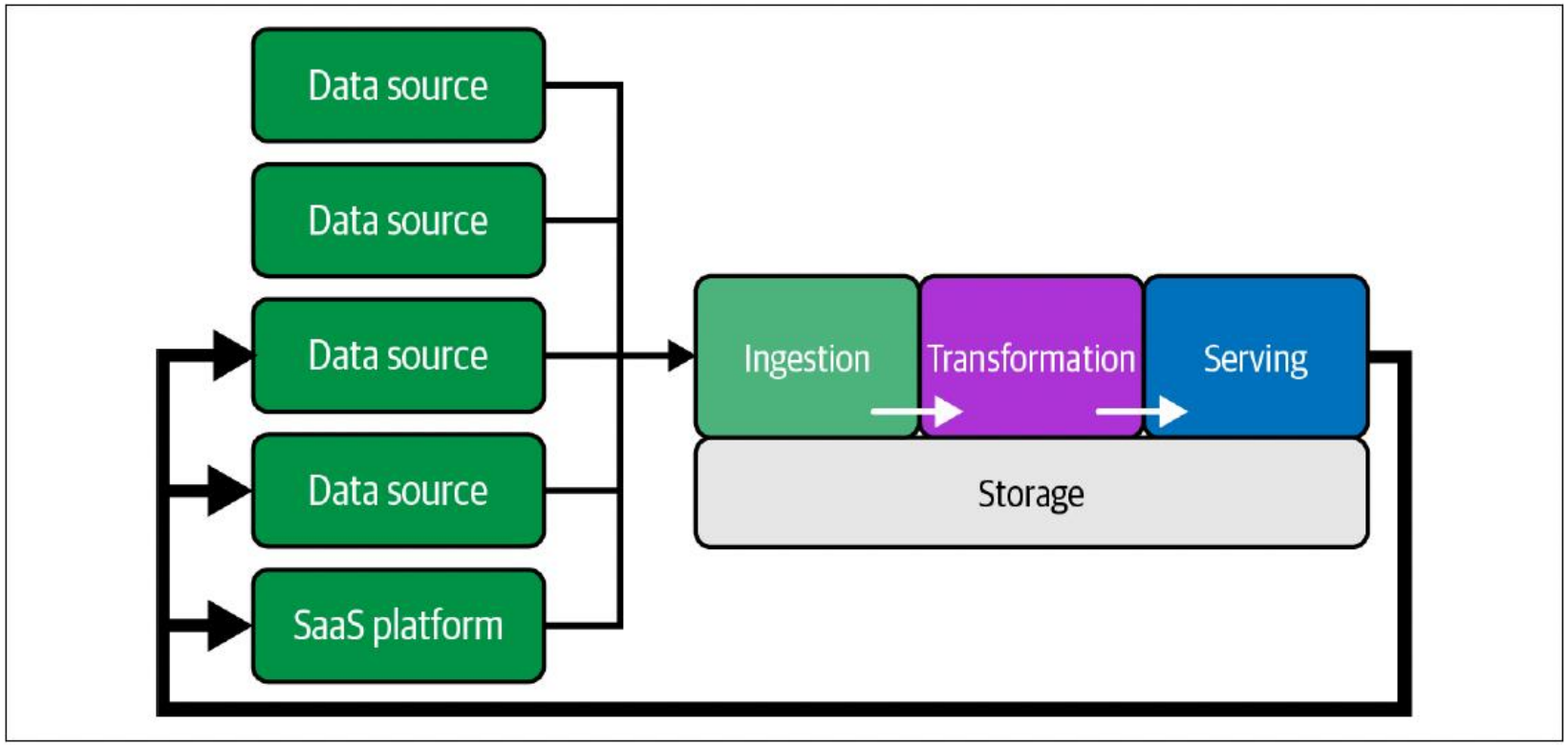
Reverse ETL sends transformed data from warehouses back into operational systems like CRMs, ad platforms, and SaaS tools.

- Once seen as an antipattern, it's now a growing practice with dedicated tools like Hightouch and Census.
- It enables teams to push metrics, predictions, and insights back into tools where actions happen.
- Common use cases include syncing model scores to marketing platforms or customer data to CRMs.
- While some argue for replacing it with event-driven patterns, the need to operationalize analytics ensures reverse ETL is here to stay.

"Reverse ETL turns insight into action, right where it matters."

Reverse ETL

Traditional ETL: source systems → data warehouse
Reverse ETL: data warehouse → business tools



THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Data Undercurrents

Major Undercurrents Across the Data Engineering Lifecycle

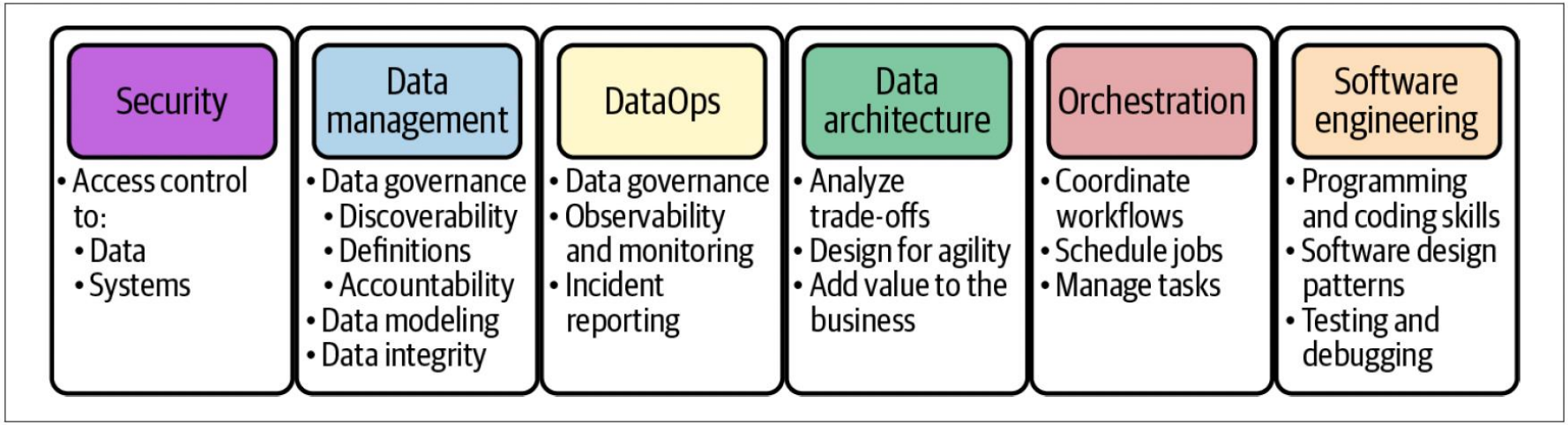
- Data engineering has evolved beyond just tools and technology.
- Modern practices focus on delivering value through structured processes and cross-functional integration.

<u>Key undercurrents shaping the lifecycle:</u>

- Security – Ensuring data protection and compliance
- Data Management – Governance, quality, and lifecycle practices
- DataOps – Agile and automated data workflows
- Data Architecture – Scalable and modular infrastructure
- Orchestration – Reliable and efficient pipeline management
- Software Engineering – Applying best practices in coding, testing, and deployment

These undercurrents support and influence every phase of the data engineering workflow.

Major Undercurrents Across the Data Engineering Lifecycle



Security – A Core Responsibility

Why It Matters
- Security is foundational in data engineering
- Most breaches come from simple mistakes or human error
- Build a culture of security across the organization

Principle of Least Privilege
- Grant users access to only what they need and no more
- Avoid admin/superuser roles unless absolutely necessary
- Helps prevent accidental damage and encourages discipline

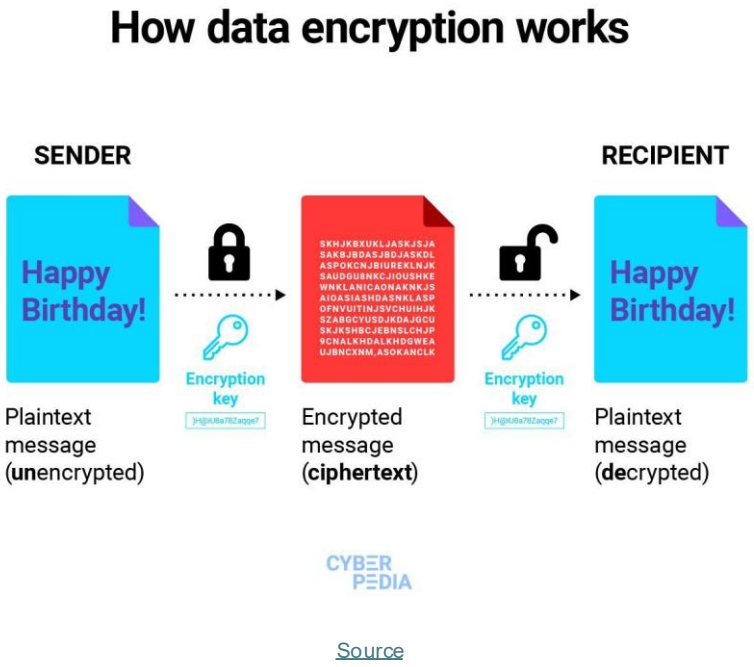
Common Mistake
Giving all users admin access is a major risk

Secure Practices for Data Engineers

Access Management:
- Limit access by role and duration
- Use time-bound permissions whenever possible

Protecting Data:
- Encrypt data in flight and at rest
- Use tokenization, masking, or obfuscation as needed
- Apply simple, enforceable access controls

Core Skills to Develop:
- IAM and role-based access
- Cloud & on-prem security basics
- Network and password policies
- Encryption methods



Source

The Rise of Data Management in Engineering

What Changed?

- Once seen as “corporate,” data management is now essential for modern data engineering.
- Simpler tools = less technical overhead → more focus on best practices.
- Practices like data governance, quality management, and metadata tracking are now adopted by organizations of all sizes.

Key Insight:

“Data management is the development, execution, and supervision of plans, policies, and practices that deliver, control, protect, and enhance the value of data.”

Why It Matters

- Data is now treated as a strategic asset, like capital or inventory.
- Without a framework, data engineers risk working in silos without alignment to business goals.

Core Facets of Data Management

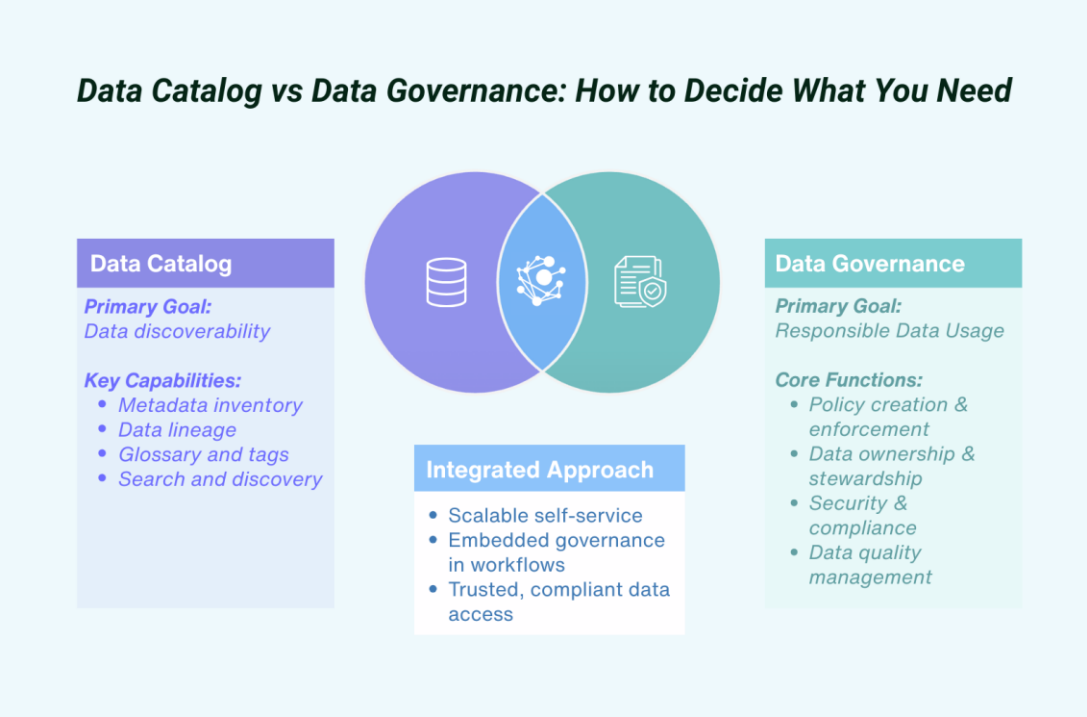
Facet	Description	Example
Governance	Establishes discoverability, ownership, and accountability for data. Defines who can access data, how it is used, and how quality, security, and compliance are enforced.	A data catalog showing dataset owners, access rules, and data quality checks.
Lineage	Tracks how data flows from source systems through transformations to downstream consumers. Supports auditing, debugging, and trust in analytical results.	Tracing a dashboard metric back through transformation jobs to the original source tables.
Storage & Operations	Provides reliable, scalable infrastructure for storing and processing data while ensuring availability, performance, and operational stability.	Cloud object storage with monitoring, backups, and automated scaling.
Integration	Enables systems to exchange data consistently and efficiently across platforms and teams. Supports batch, streaming, and API-based data movement.	Ingesting data from SaaS tools into a warehouse using scheduled pipelines.
Modeling & Design	Structures data to improve clarity, performance, and usability for analytics and applications. Emphasizes well-defined schemas and reusable data models.	A star schema designed to support fast analytical queries.
Lifecycle Management	Manages data from creation through retention, archival, and deletion in accordance with policy and cost constraints.	Automatically deleting logs after a defined retention period.
Advanced Systems	Supports analytics, machine learning, and AI workloads at scale using specialized data platforms and compute engines.	A feature store serving real-time features to a production ML model.
Ethics & Privacy	Ensures responsible data use by addressing consent, fairness, security, and regulatory compliance throughout the data lifecycle.	Masking personally identifiable information in analytics datasets.

Data Governance – A Strategic Foundation

Definition
"A data management function to ensure the quality, integrity, security, and usability of organizational data." – Data Governance: The Definitive Guide

Expanded View
- Aligns people, processes, and technologies
- Ensures data value is maximized while applying appropriate controls
- Should be intentional, not ad hoc, to avoid issues like: untrusted reports, security breaches, and misguided decision-making

Real-World Consequence
Poor governance = wasted time, inaccurate insights, and organizational confusion.



Source

Core Elements of Data Governance

Three Pillars of Effective Governance

- Discoverability: Data should be easy to locate, understand, and access across the organization.
- Security: Data must be protected from unauthorized access and misuse through appropriate controls.
- Accountability: Clear ownership and responsibility must be established to maintain data accuracy and integrity.

Strong governance ensures trust in data, supports compliance, and enables informed decision-making.

What is Metadata and Why It Matters

Definition

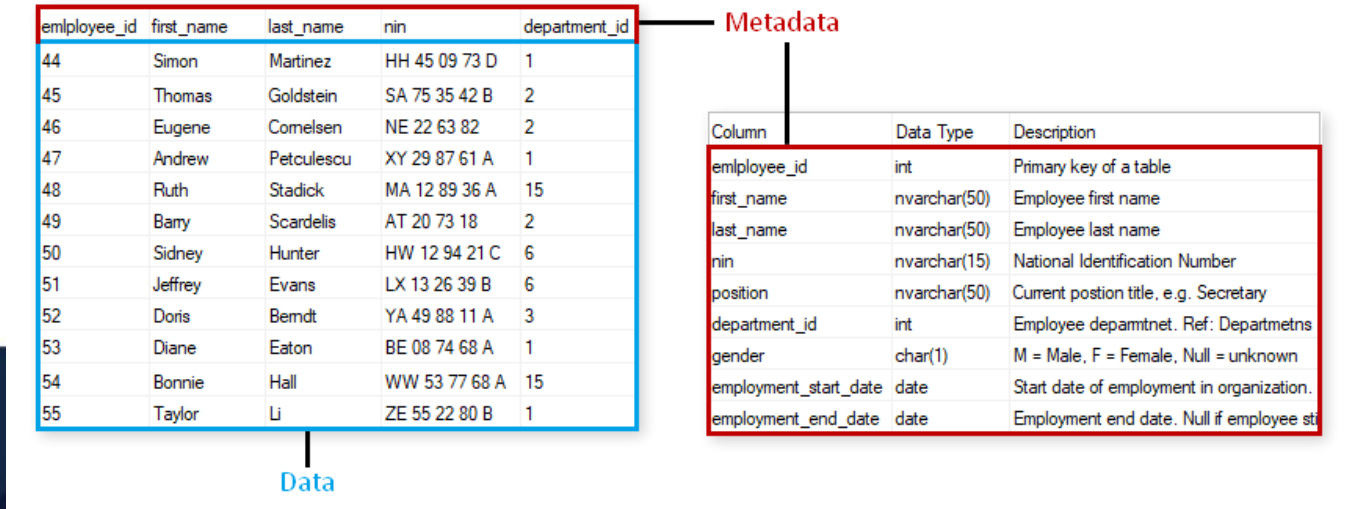
Metadata is *"data about data."* It provides the essential context for understanding, organizing, and using data effectively across systems.

Role in Data Engineering

- Enables discoverability, governance, and pipeline design
- Critical for ensuring data quality, reproducibility, and accountability
- Without it, data systems become opaque and difficult to manage or trust

Metadata in Action - Answers key questions:

- What does this data mean?
- Where did it come from?
- Who owns or uses it?



Source

How Metadata is Collected and Managed

Two Sources of Metadata

- Autogenerated: Created by systems (schemas, logs, pipeline steps)
- Human-generated: Created by people (documentation, ownership, definitions)

The Social Layer of Metadata

- Metadata reflects both technical structure and organizational context
- Tools like Airbnb's Data portal demonstrate how metadata systems must capture:
  - Data owners
  - Usage patterns
  - Domain knowledge

Challenges

- Interoperability and standardization remain immature
- Even the best tools need human oversight and cultural buy-in

Four Core Categories of Metadata

1. Business Metadata
Defines how data is used in context…ownership, business rules, and definitions.
Used to answer nontechnical questions like: What counts as a customer?

2. Technical Metadata
Schema, data models, lineage, pipeline configs.
Helps engineers build, monitor, and troubleshoot data workflows.

3. Operational Metadata
Job status, logs, runtime metrics, errors.
Used to evaluate process performance and health.

4. Reference Metadata
Classifies other data, e.g., codes, units, calendars.
Provides stable, shared context across systems and reports.

Why Metadata is Foundational

Metadata Powers the Data Lifecycle

- Design: Helps structure pipelines and models
- Execution: Supports traceability and monitoring
- Governance: Provides transparency, trust, and compliance
- Collaboration: Aligns technical and business teams

When used effectively, it transforms raw data into trusted, usable, and valuable assets across the organization.

Metadata Type	Definition	Examples
Descriptive	Defines or describes an information resource to aid identification, recovery, and retrieval at any and all levels of aggregation.	• Publication-level metadata • Citation metadata • Subject indexing • Linking metadata
Technical	Describes objective technical information about an information resource.	• File size • Pixel height • Duration
Administrative	Supports the general management and use of an information resource.	• Identification metadata • Content lifecycle metadata • Versioning metadata
Structural	Defines what the component objects are and how they relate to each other.	• Product definition metadata • Product organization, assembly metadata
Rights	Supports the management of an information resource's intellectual property.	• Geographic scope metadata • Timeframe metadata • Rights holder metadata
Presentation	Defines how information will be formatted for a particular object.	• Tagging for browser display

Source

Data Accountability

Definition
Data accountability is the assignment of ownership over a portion of data to an individual who ensures its governance and quality.

Who Is Accountable
The responsible person may be a product manager, software engineer, or other stakeholder, not necessarily a data engineer. Their role is to coordinate governance across all involved parties.

Scope of Accountability
Can apply at various levels: entire tables or log streams, or individual fields (e.g., a customer ID used across systems).

Why It Matters
Without clear accountability, data quality is difficult to manage. Defined ownership improves consistency, governance, and trust in data assets.

Data Quality

“Can I trust this data?” —> Everyone in the business

Definition

Data quality is the optimization of data toward its expected state, based on business-defined standards. It reflects how closely the data matches what is intended or required.

Data Engineer’s Role

Validate data through quality tests
Ensure conformance to schema
Monitor completeness, accuracy, and precision across the lifecycle

Three Key Dimensions

- Accuracy – Are values correct, valid, and deduplicated?
- Completeness – Are all necessary fields present and filled?
- Timeliness – Is the data available when it's needed?

Data Modeling and Design in Data Engineering

Definition
The process of shaping raw data into usable structures for analytics and decision-making.

Why It Matters
- Enables reliable insights from complex, varied data
- Poor modeling leads to unusable data ("write once, read never")

Modern Realities
- Happens across roles, from firmware to APIs to data warehouses
- Tools like cloud warehouses and Spark support both structured and semi-structured data
- Common design patterns: Kimball, Inmon, Data Vault

Data modeling remains essential for scalable, trustworthy, and interpretable systems despite growing complexity.

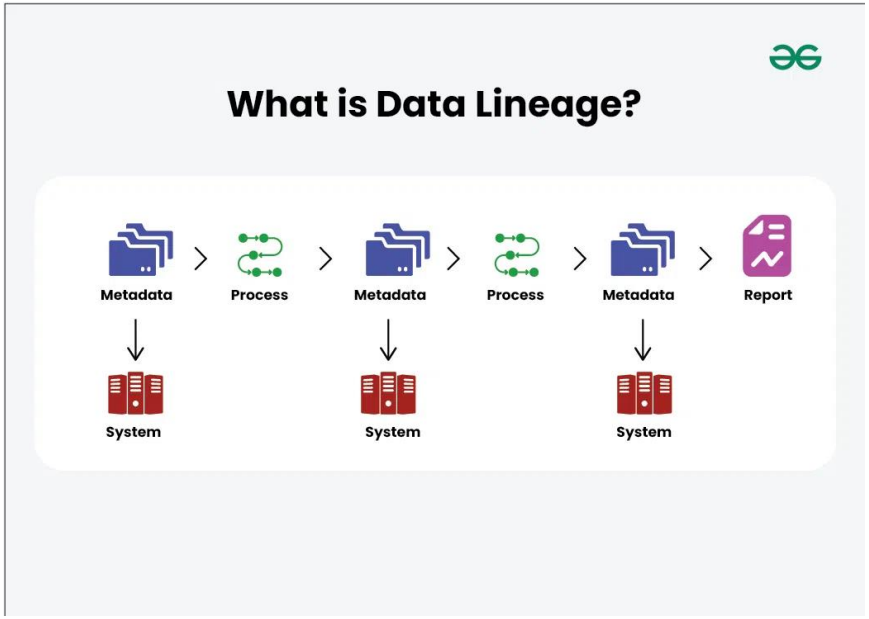
Data Lineage - Tracking the Data Lifecycle

Definition
Tracks how data moves, changes, and is processed across systems over time.

Why It Matters
- Provides an audit trail
- Aids debugging, error tracking, and accountability
- Supports compliance and data deletion requests

Regulated Industries
Finance, Healthcare, Pharma, Retail, and Tech (e.g., GDPR, HIPAA, SOX)

Modern Approach
Related to Data Observability Driven Development (DODD)...monitoring data from development to production.



Source

Data integration and interoperability

Definition
The process of connecting data across diverse tools, platforms, and systems.

Why It Matters
- Modern data environments are heterogeneous and cloud-based
- Integration now relies on APIs, not just direct database connections

Typical Workflow Example
Pull from Salesforce → Store in S3 → Load into Snowflake → Query via API → Export to S3 for Spark

Main Takeaway
While tools simplify system interactions, growing pipeline complexity demands orchestration, not just scripts.

Data Lifecycle Management

Definition
Managing data from creation through archival and deletion to optimize cost, compliance, and usability.

Why It Matters
- Cloud storage = ongoing costs → archiving becomes essential
- Privacy laws (e.g., GDPR, CCPA) require active data deletion

Modern Practices
- Cloud providers offer archival storage with policy controls
- SQL-based deletion is now common in cloud warehouses
- Tools like Delta Lake and Hive ACID support scalable deletions

Ethics and Privacy in Data Engineering

Why It Matters
Data impacts real people. Ethics and privacy are now central (not optional) in the data lifecycle.

Key Responsibilities
- Mask PII and sensitive information
- Monitor and mitigate bias in data
- Ensure compliance with regulations like GDPR and CCPA

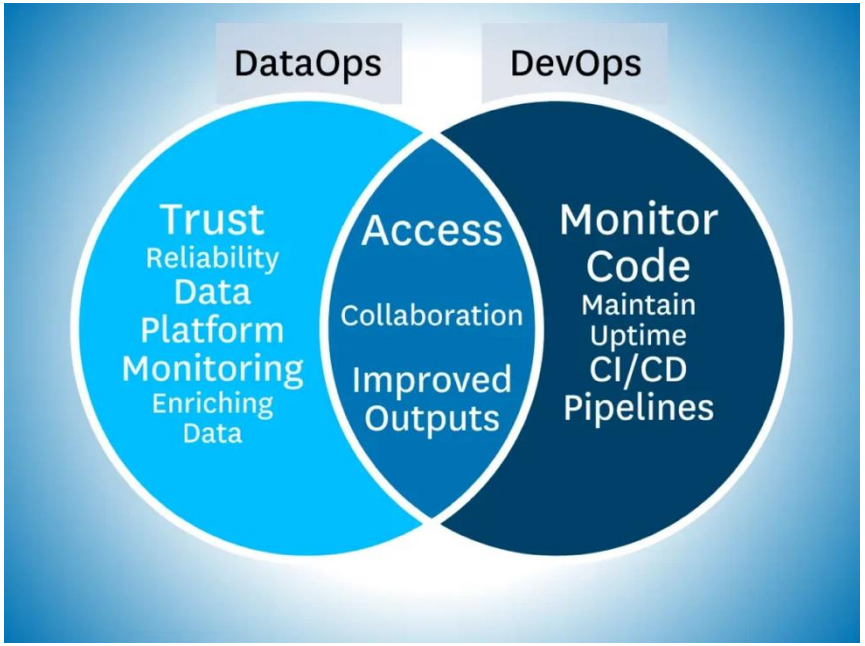
Cultural Imperative
Data engineers must act responsibly, even when no one is watching. Ethical data practices build long-term trust and accountability.

DataOps

Definition
DataOps adapts Agile, DevOps, and lean practices to improve the delivery of data products.

Key Focus
- Rapid iteration and innovation
- High data quality, low error rates
- Collaboration across teams
- Transparent monitoring and feedback

Cultural Foundation
Effective DataOps starts with team habits such as communication, shared learning, and alignment with business needs.

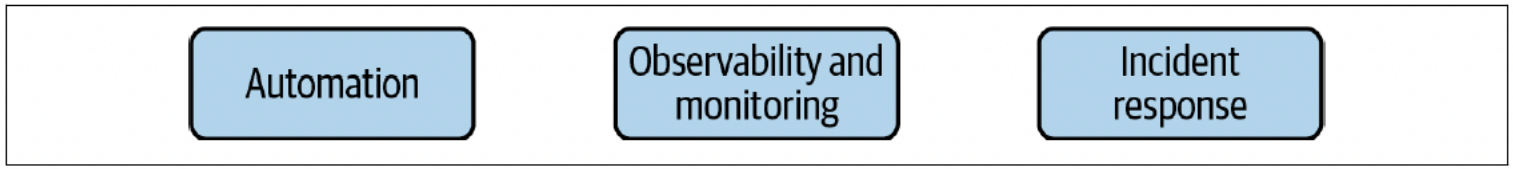


Source

Applying DataOps

Pillars of DataOps

- Automation – Build, test, and deploy pipelines
- Monitoring & Observability – Track system performance
- Incident Response – Quickly resolve failures



Adoption Tips

- Start with observability
- Gradually introduce automation and response workflows
- In mature orgs, work with existing DataOps teams

Data Architecture

Definition
Data architecture defines the structure and strategy of data systems to meet current and future organizational needs.

Why It Matters
- Guides how data is captured, stored, transformed, and served
- Balances performance, cost, and operational simplicity
- Adapts to evolving tools, use cases, and business requirements

Role of the Data Engineer
Understand business needs, apply design trade-offs, and collaborate with data architects to implement scalable and efficient solutions.

Orchestration

Definition
Orchestration is the process of coordinating data jobs to run efficiently, reliably, and in the right order, often using metadata and dependencies.

Why It Matters
- Central to the data platform and data lifecycle
- Goes beyond simple scheduling (e.g., cron)
- Manages complex job dependencies using DAGs
- Enables automation, monitoring, and alerting

Core Capabilities
- Job sequencing based on conditions, not just time
- Continuous monitoring and error handling
- Backfilling, visualizations, and alerting
- Supports data SLAs (e.g., alert if not complete by 10 a.m.)

Orchestration - Tools, Trends, and Evolution

Historical Context
- Early tools: Apache Oozie (Hadoop-specific), Facebook's Dataswarm
- Game changer: Airflow (2014, open source, Python-based)

Modern Orchestration Tools
- Airflow – Widely adopted, flexible, and extensible
- Prefect – Focuses on DAG portability and testing
- Dagster – Emphasizes developer experience
- Argo – Built for Kubernetes
- Metaflow – Netflix tool for data science workflows

Streaming Context
Orchestration applies to batch workflows. For streaming data, tools like Pulsar are advancing streaming DAGs, which remain more complex to manage.

Software Engineering in Data Engineering

Core Role
Software engineering remains foundational to data engineering, even as tools become more abstract.

Key Practices
- Write robust data processing code (SQL, Spark, Beam)
- Apply testing methodologies (unit, integration, regression, end-to-end)
- Handle complex streaming tasks like joins and windowing
- Build pipelines as code with orchestration engines (e.g., Airflow, Prefect)

Why It Matters
Data engineers solve general-purpose problems and must write custom logic when tools fall short, especially when handling novel data sources or edge cases.

Infrastructure & Pipelines as Code

Infrastructure as Code (IaC)

- Automates cloud setup using frameworks like Terraform, Helm, and Kubernetes
- Enables repeatability, version control, and scaling in modern environments
- Critical for managing cloud-native tools like Databricks or EMR

Pipelines as Code

- Define workflows using code (typically Python)
- Orchestration tools use this to manage DAG execution, dependencies, and retries
- Central to modern orchestration and DataOps practices

Takeaway

IaC and pipelines as code bring DevOps principles into data engineering which is ensuring efficiency, traceability, and automation.

Evolving Toolsets and Open Source Contribution

Framework Explosion
- Shift from low-level MapReduce to high-level frameworks (Airflow, Spark, Flink)
- Engineers contribute to open source to extend functionality and improve community tools

Choosing Wisely
- Don’t reinvent the wheel. You should evaluate total cost ownership (TCO) and existing solutions before building in-house
- Consider extensibility, community support, and alignment with use case

Streaming & Edge Cases
- Streaming is still complex. Tools like Pulsar, Beam, and Flink help
- Event-driven architectures (Lambda, GCF) introduce new engineering challenges

Bottom Line
Strong software engineering skills help data engineers adapt to evolving tech, build scalable systems, and solve problems tools alone can’t handle.

Conclusion

Beyond Tools
Data engineering is more than just technology. It's about managing the entire data lifecycle with purpose and strategy.

Lifecycle Stages
Generation → Storage → Ingestion → Transformation → Serving

Undercurrents Across the Lifecycle
Security, Data Management, DataOps, Data Architecture, Orchestration, Software Engineering

Core Goals of the Data Engineer
- Maximize data value
- Optimize ROI and reduce costs
- Minimize risk through secure, high-quality, and well-managed data systems

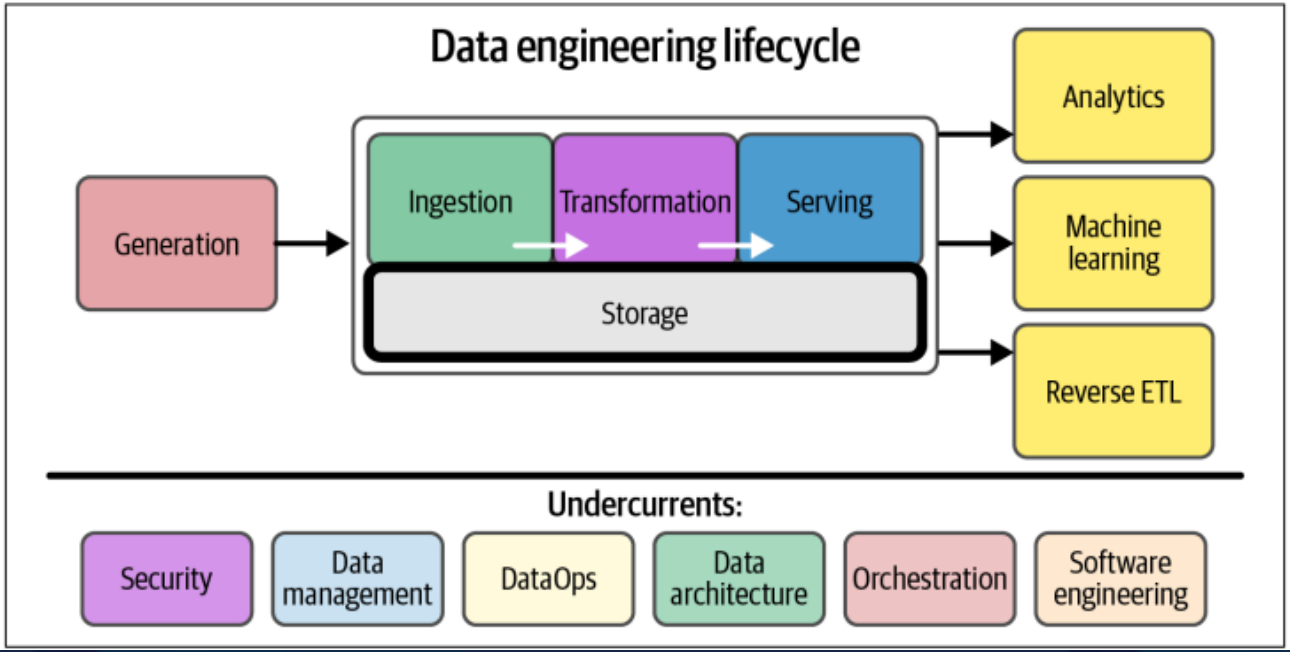
THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Data Storage: Structures & Interaction

Data Storage 101

- Data sources
- Data formats
- Data models
- Data storage engines & processing



Data Sources

Data Source Type	Description	Example
User Generated	Data created directly by users through interactions, inputs, or content creation. Often unstructured or semi-structured and generated at high volume.	Form submissions, uploaded files, user reviews, click events.
Systems Generated	Data automatically produced by software systems as part of normal operation. Typically structured and generated continuously.	Application logs, system metrics, event streams.
Internal Databases	Data maintained within an organization to support core business operations and processes. Usually well-structured and governed.	User accounts, inventory tables, customer relationship management (CRM) records.
Third-Party Data	Data sourced from external providers or partners to enrich internal datasets or enable broader analysis. Often governed by licensing or usage agreements.	Weather data APIs, demographic datasets, market research feeds.

Big Data Characteristics

- The 7 V's of Big Data are:
  - **Volume** – There is a massive amount of data generated every second.
  - **Velocity** – The speed at which data is generated, collected, and analyzed
  - **Variety** – The different types of data: structured, semi-structured, unstructured
  - **Veracity** – Trustworthiness in terms of quality and accuracy
  - Value – Ability to extract useful insights from data
  - Variability – Inconsistency in data meaning, structure, or arrival rates
  - Visualization – Ability to represent data clearly for understanding and insight

* Bold are the original; with the others added more recently

Data Sources

Aspect	User-Generated Data	System-Generated Data
Source	Created directly by users through interactions or inputs	Automatically produced by systems during operation
Typical Content	User inputs, free-text fields, uploads, form entries	Logs, metadata, metrics, predictions, events
Structure & Formatting	Often inconsistent or malformed	Generally consistent and easier to standardize
Processing Urgency	Usually acceptable to process periodically	Often needs to be processed as soon as possible
Time Sensitivity	Lower, unless tied to real-time features	High, especially for monitoring and issue detection
Growth Pattern	Can grow quickly with user adoption	Can grow very large very quickly due to high volume
Common Tooling	Data cleaning and validation pipelines	Log and observability tools (Logstash, Datadog, Logz, etc.)

Users' behavioral data (clicks, time spent, etc.) is often system-generated but is considered **user data**

How to store your data?

Storing your data is only interesting if you want to access it later.

Data engineering pipelines constantly move data between:

- Applications
- Databases
- Files
- Data lakes
- Data warehouses
- Streaming systems
- **Storage systems don't understand Python objects, Java classes, or Pandas DataFrames.**
  …They understand bytes → so we serialize.

● **Serialization** = converting in-memory data (objects, rows, records) into a storable or transmittable format

● **Deserialization** = converting that stored/transmitted data back into usable in-memory structures

Data Formats

- Data formats are agreed upon standards to serialize your data so that it can be transmitted & reconstructed later
- Questions to consider:
  - How to store multimodal data?
    - `{'image': [[200,155,0], [255,255,255], ...], 'label': 'car', 'id': 1}`
  - Access patterns
    - How frequently the data will be accessed?
  - The hardware the data will be run on
    - Complex ML models on TPU/GPU/CPU

Data formats

Row-major

Column-major

Format	Binary/Text	Human-readable	Example use cases
JSON	Text	Yes	Everywhere
CSV	Text	Yes	Everywhere
Parquet	Binary	No	Hadoop, Amazon Redshift
Avro	Binary primary	No	Hadoop
Protobuf	Binary primary	No	Google, TensorFlow (TFRecord)
Pickle	Binary	No	Python, PyTorch serialization

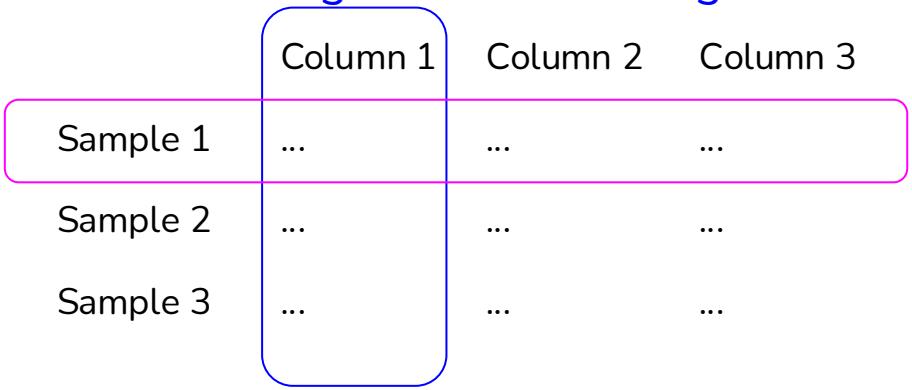
Row-major vs. column-major

Column-major:

- stored and retrieved column-by-column
- good for accessing features

Row-major:

- stored and retrieved row-by-row
- good for accessing samples



Question for the class: how do Pandas and Numpy stack up?

Row-major vs. column-major: DataFrame vs. ndarray

Aspect	NumPy	Pandas
Primary abstraction	N-dimensional array	Table (DataFrame)
Memory layout	Row-major (default)	Column-based arrays
Optimized for	Math, linear algebra	Data analysis, ETL
Best access pattern	Row-wise / vectorized	Column-wise
Typical use	Models, tensors	Datasets, features

```
# Get the column `date`, 1000 loops
%timeit -n1000 df["Date"]

# Get the first row, 1000 loops
%timeit -n1000 df.iloc[0]
```

```
1.78 µs ± 167 ns per loop (mean ± std. dev. of 7 runs, 1000 loops each)
145 µs ± 9.41 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

```
df_np = df.to_numpy()
%timeit -n1000 df_np[0]
%timeit -n1000 df_np[:,0]
```

```
147 ns ± 1.54 ns per loop (mean ± std. dev. of 7 runs, 1000 loops each)
204 ns ± 0.678 ns per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

Pandas Demo

Text vs. binary formats

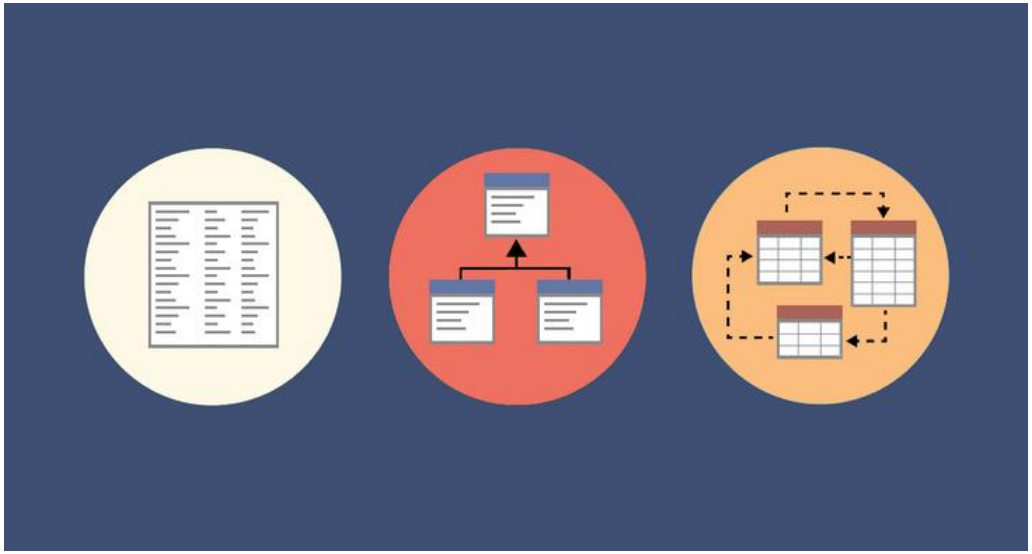
	Text files	**Binary files**
Examples	CSV, JSON	Parquet
Pros	Human readable	Compact; not human readable
Store the number *1000000*?	7 characters -> 7 bytes	If stored as int32, only 4 bytes

You can unload the result of an Amazon Redshift query to your Amazon S3 data lake in Apache Parquet, an efficient open columnar storage format for analytics. Parquet format is up to 2x faster to unload and consumes up to 6x less storage in Amazon S3, compared with text formats. This enables you to save data transformation and enrichment you have done in

Data models

What is a Data Model?

- Describes how data is structured, stored, and related
- Defines how applications and analytics interact with data
- Shapes performance, scalability, and query patterns



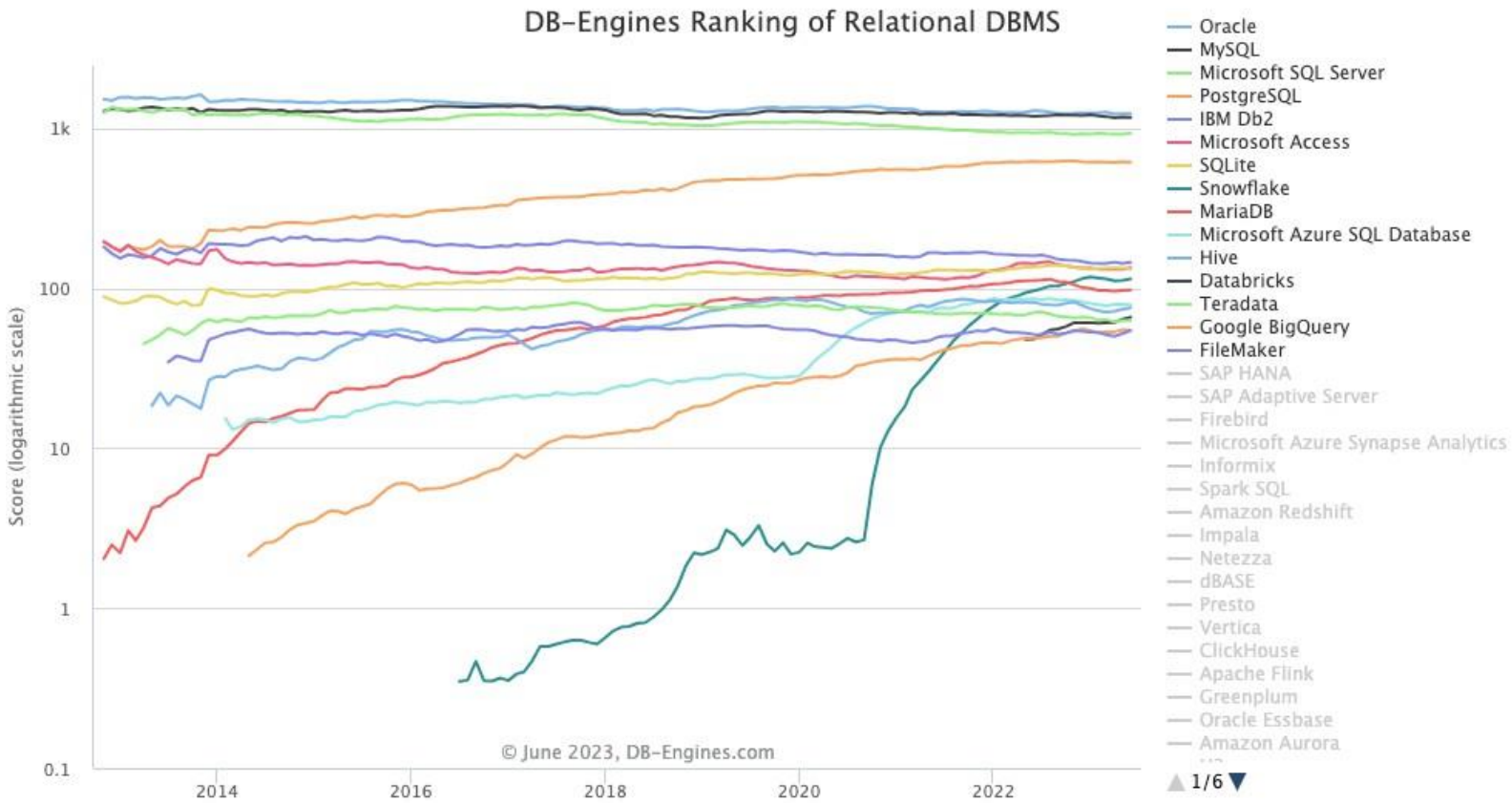
Data Models

Two Main Data Model Paradigms

- **Relational Model (SQL)**
  - Represents data in tables (rows and columns)
  - Uses primary keys and foreign keys to model relationships
  - Strong schema enforcement and data integrity
  - Originated in the 1970s and remains widely used today
  - Common systems:
    - Oracle
    - MySQL
    - PostgreSQL
    - Microsoft SQL Server
- **NoSQL Models**
  - Designed for flexibility and scale
  - Schema-less or schema-light
  - Optimized for distributed systems
  - Common types:
    - Key-value
    - Document
    - Columns-Family
    - Graph

How we tend to think about data:

- Humans naturally think in tabular structures
- Relationships are expressed via connectivity between tables
- This mental model aligns closely with relational databases



https://www.linkedin.com/pulse/database-engine-ranking-popularity-2023-tamer-khraisha-ph-d-/

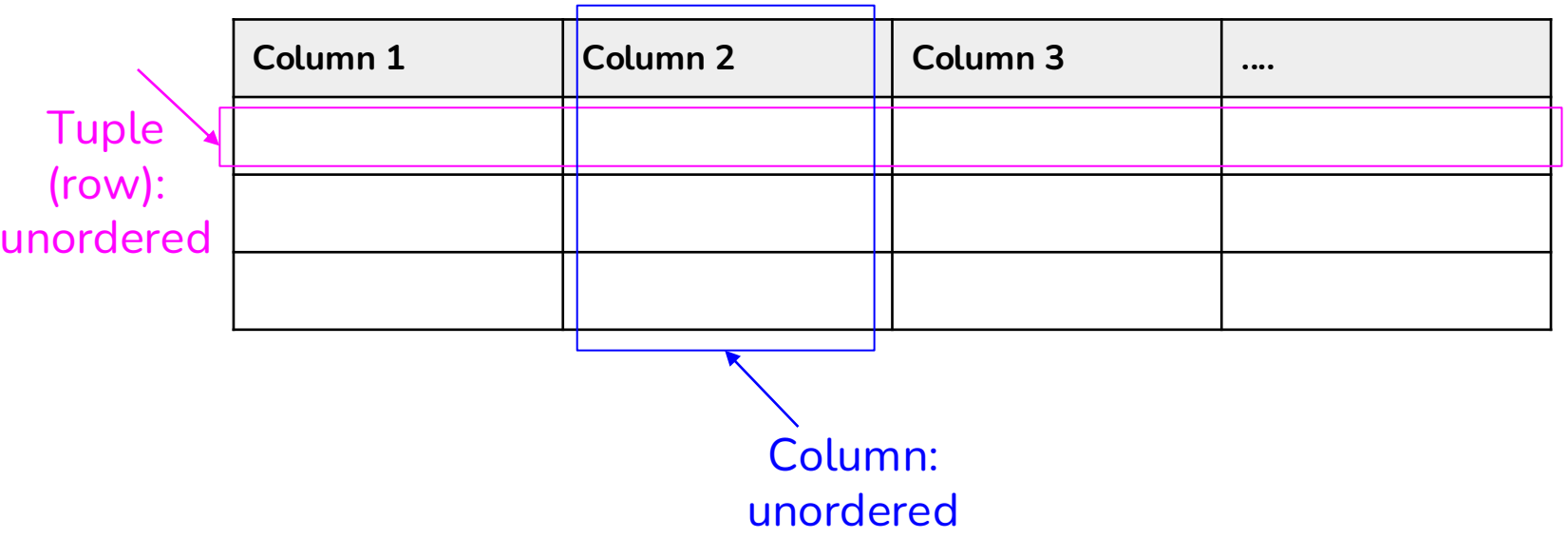
What is means to be relational

Relational refers to a data model in which:

- Data is organized into **tables (relations)**
- Each table consists of **rows (tuples)** and **columns (attributes)**
- Relationships between tables are defined using **keys**
    - **Primary keys** uniquely identify rows
    - **Foreign keys** link related tables
- Data integrity is enforced through **constraints** and **schemas**

Relational model (est. 1970)

- Similar to SQL model
- Formats: CSV, Parquet



Normalization

Normalization

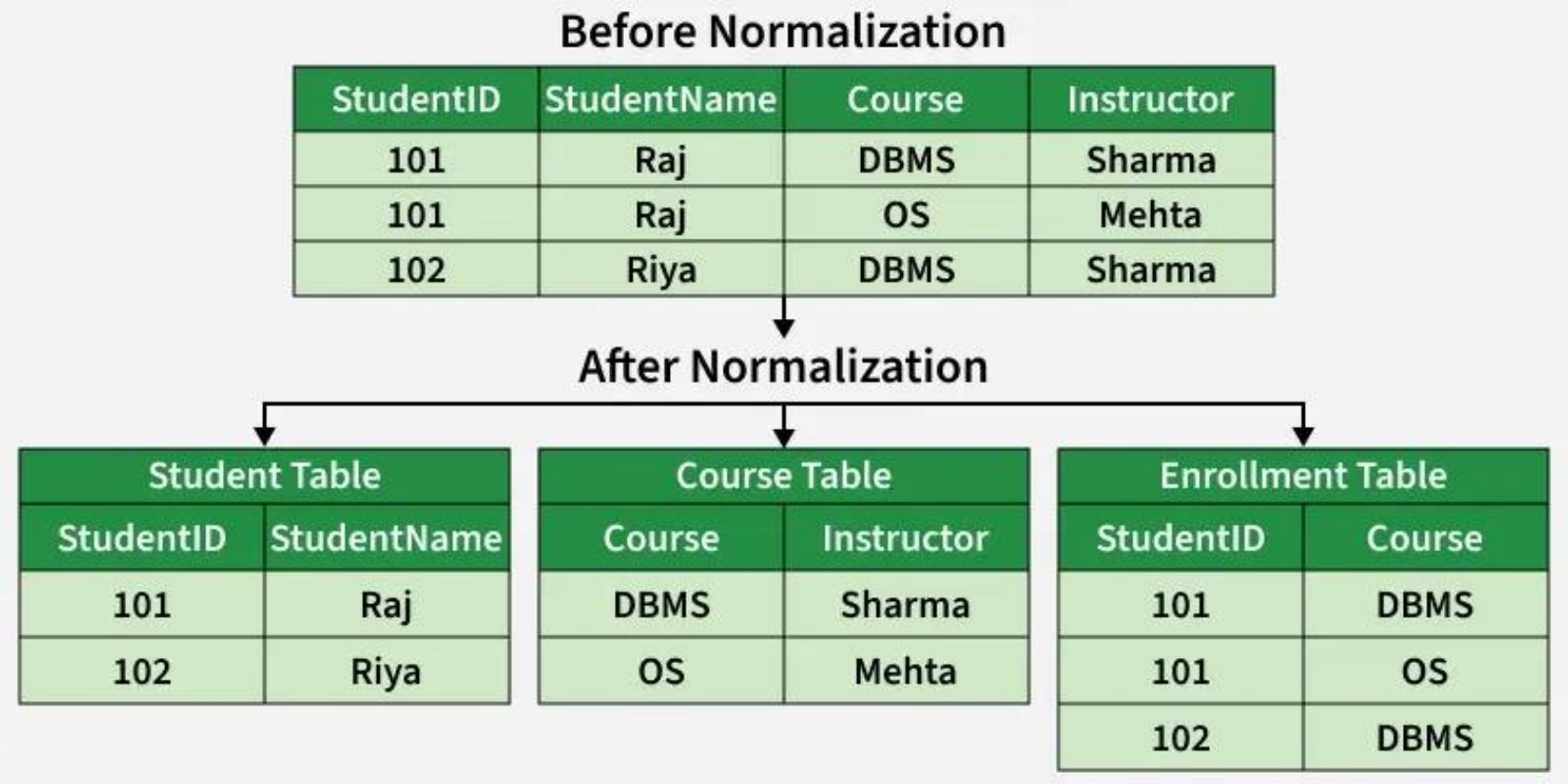
- Organizing data into separate, related tables to reduce redundancy and improve data integrity.

Denormalization

- Intentionally combining or duplicating data across tables to improve read performance and simplify queries.

Approach	Pros	Cons
Normalization	• Reduces data duplication • Improves consistency and integrity • Easier updates	• Requires more joins • Slower read queries
Denormalization	• Faster reads • Fewer joins • Simpler queries	• Data duplication • Risk of inconsistency • More complex updates

Normalization



Source

Relational Model & SQL Model

- SQL model slightly differs from relational model
  - e.g. SQL tables can contain row duplicates. True relations can't.
- SQL is a query language
  - How to specify the data that you want from a database
  - Structured Query Language (SQL)
- SQL is declarative
  - You tell the data system what you want
  - It's up to the system to figure out how to execute
    - Query optimization

SQL

- SQL is an essential data scientists' tool

LEARN SQL!

Problems with SQL

- What if we add a new column?
- What if we change a column type?

Limitations of SQL (Relational Databases)

- Rigid schema defined upfront
- Schema changes are costly (adding columns, changing data types)
  - Adding columns may require touching or rewriting large tables
  - Changing data types can require full table rewrites and data conversion
  - Changes can cause table locks, downtime, or blocked reads/writes
- Migrations required, often involving table locks or downtime
- Difficult schema evolution when data changes frequently
- Joins can become expensive at large scale
- Horizontal scaling is complex and often application-dependent

SQL databases optimize for structure and consistency, not rapid change.

Data Storage – NoSQL

1. **NoSQL databases**: NoSQL databases are non-relational databases that can handle unstructured, semi-structured, or differently structured data. They are designed for horizontal scalability and high availability. NoSQL databases can be further classified into several categories:

    - Document-oriented databases (e.g., MongoDB, Couchbase)
    - Key-value stores (e.g., Redis, Amazon DynamoDB)
    - Column-family stores (e.g., Apache Cassandra, HBase)
    - Graph databases (e.g., Neo4j, Amazon Neptune)

2. **Time-series databases**: These databases are designed to store and manage time-series data, which consists of data points indexed by time. Time-series databases are optimized for high write and query performance, data retention, and handling large volumes of time-stamped data. Examples include InfluxDB, OpenTSDB, and TimescaleDB.

Data Storage - NoSQL

3. **Object storage**: Object storage is a data storage architecture that manages data as objects, rather than files or blocks. It is designed for high scalability, durability, and cost-efficiency, making it suitable for storing and managing large amounts of unstructured data, such as multimedia files, backups, and archives. Examples include Amazon S3, Google Cloud Storage, and Microsoft Azure Blob Storage.

4. **Distributed file systems**: Distributed file systems allow files to be stored across multiple servers or nodes, providing high availability, fault tolerance, and horizontal scalability. These systems are particularly useful for managing large files or datasets that need to be accessed and processed concurrently by multiple users or applications. Examples include Hadoop Distributed FileSystem (HDFS), GlusterFS, and Ceph.

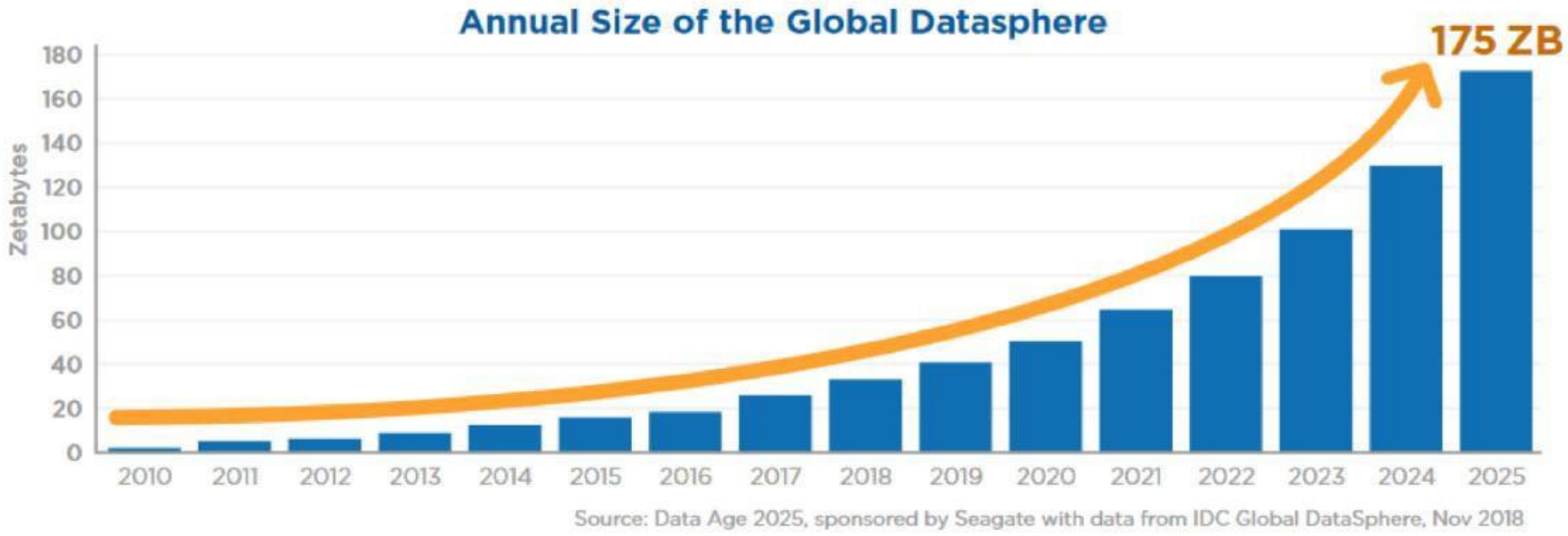
Data Storage - NoSQL

5. **In-memory data stores**: In-memory data stores store data in the main memory (RAM) of a computer or a cluster of computers, providing low-latency access and high throughput for read and write operations. They are often used for caching, real-time data processing, and analytics. Examples include Redis (in-memory key-value store) and Apache Ignite (in-memory data grid).

- **SAP HANA** is one of the latest inventions/stores which is used for both reporting and transactional data store. (Oracle/other companies have other products, like times 10….etc)

6. **File Based**: e.g. Splunk and Elasticsearch are both powerful platforms used for searching, analyzing, and visualizing large volumes of data. Search Processing Language is splunk's language to query the indexed/on the fly indexed data.

Data in the digital world



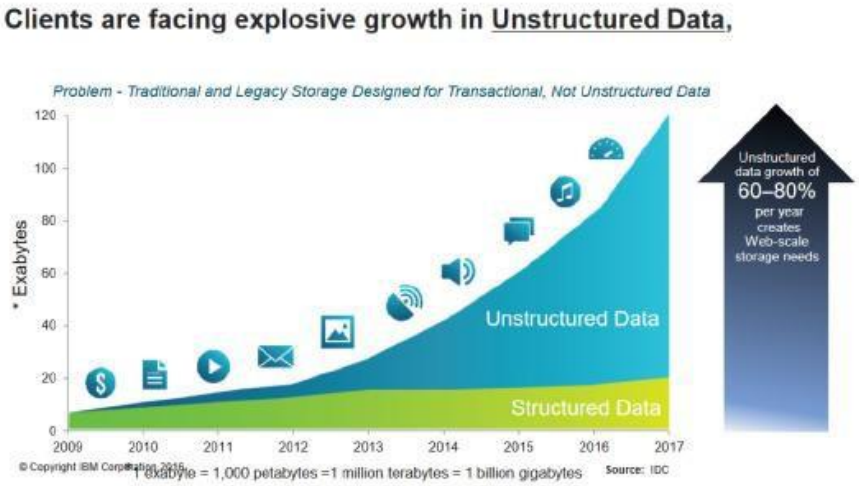
1 Zetta-byte = 1,000,000,000 terabytes (TB)

Src:: https://hexanika.com/big-data/

Data Growth

UNSTRUCTURED DATA:
80% of the worldwide data that will be generated by 2025 will be unstructured because of:

- Heterogeneous Sources: Due to data from multiple sources being pushed into a centralized data repository (Data Lake / Data Warehouse)
- Data Quality: Multiple file formats & types
- Thirdly, Governance & Auditability: Increased data management costs and Lack of business insights & analytics



Src:: https://hexanika.com/big-data/

NoSQL -> Not Only SQL

- Document model
  - Central concept: document
  - Relationships between documents are rare
- Graph model
  - Central concept: graph (nodes & edges)
  - Relationships are the priority

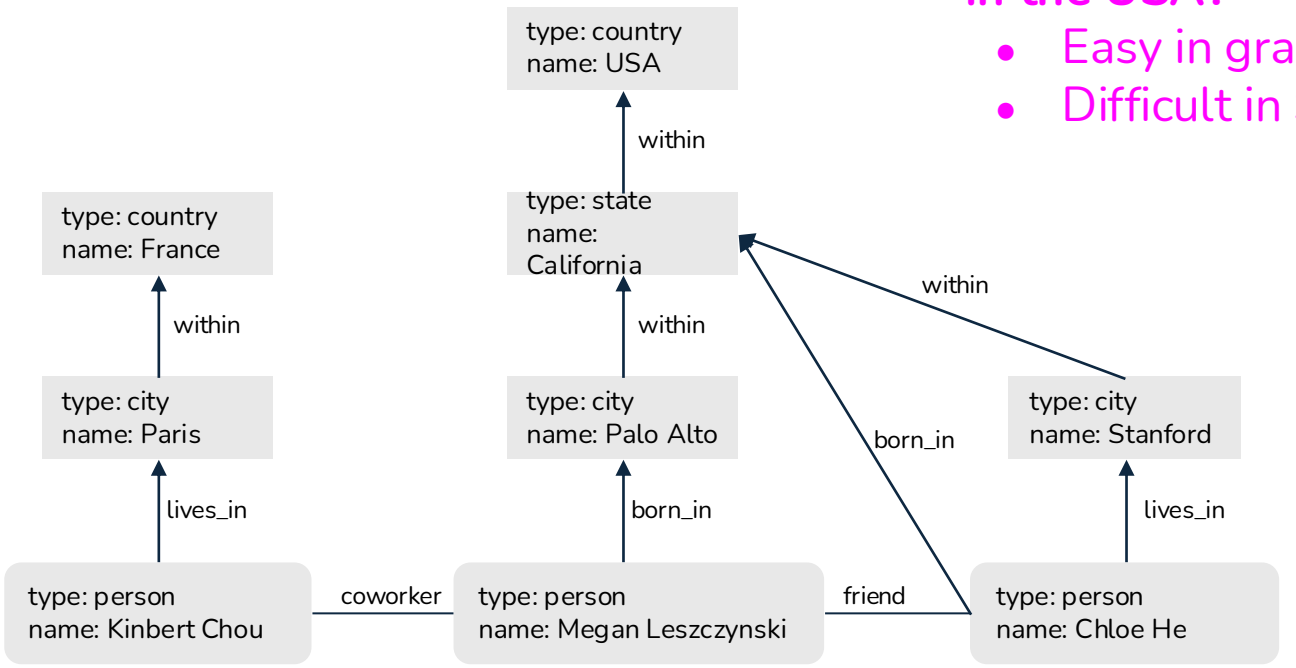
- Book data in the document model
- Each book is a document

```
# Document 1: harry_potter.json
{
    "Title": "Harry Potter",
    "Author": "J.K. Rowling",
    "Publisher": "Banana Press",
    "Country": "UK",
    "Sold as": [
        {"Format": "Paperback", "Price": "$20"},
        {"Format": "E-book", "Price": "$10"}
    ]
}

# Document 2: sherlock_holmes.json
{
    "Title": "Sherlock Holmes",
    "Author": "Conan Doyle",
    "Publisher": "Guava Press",
    "Country": "US",
    "Sold as": [
        {"Format": "Paperback", "Price": "$30"},
        {"Format": "E-book", "Price": "$15"}
    ]
}

# Document 3: the_hobbit.json
{
    "Title": "The Hobbit",
    "Author": "J.R.R. Tolkien",
    "Publisher": "Banana Press",
    "Country": "UK",
    "Sold as": [
        {"Format": "Paperback", "Price": "$30"},
    ]
}
```

Graph model

Query: show me everyone who was born in the USA?

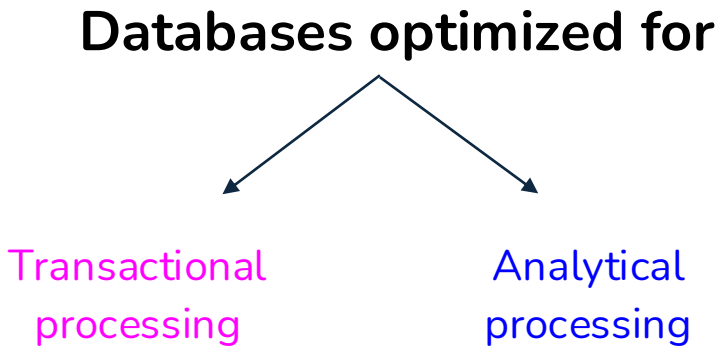
- Easy in graph
- Difficult in SQL



Structured vs. unstructured data

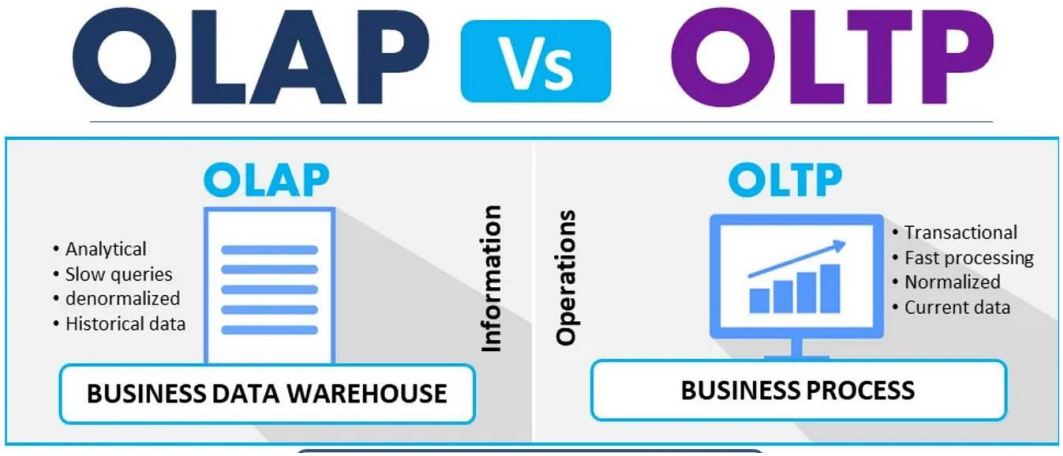
Structured	Unstructured
Schema clearly defined	Whatever
Easy to search and analyze	Fast arrival (e.g. no need to clean up first)
Can only handle data with specific schema	Can handle data from any source
Schema changes will cause a lot of trouble	No need to worry about schema changes
Data warehouses	Data lakes
Structure is assumed at write	Structure is assumed at read

Data Storage Engines & Processing



OnLine Transaction Processing (OLTP)

- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
- Operations:
  - Insert when generated
  - Occasional update/delete



Source

OnLine Transaction Processing

- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
- Operations:
  - Inserted when generated
  - Occasional update/delete
- Requirements
  - Low latency
  - High availability

OnLine Transaction Processing

- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
- Operations:
  - Inserted when generated
  - Occasional update/delete
- Requirements
  - Low latency
  - High availability
  - Full ACID not always necessary

ACID describes a set of properties that guarantee reliable transactions in databases.

- **Atomicity** – A transaction is all-or-nothing
- **Consistency** – The database moves from one valid state to another
- **Isolation** – Concurrent transactions do not interfere with each other
- **Durability** – Once committed, data persists even after failures

OnLine Transaction Processing

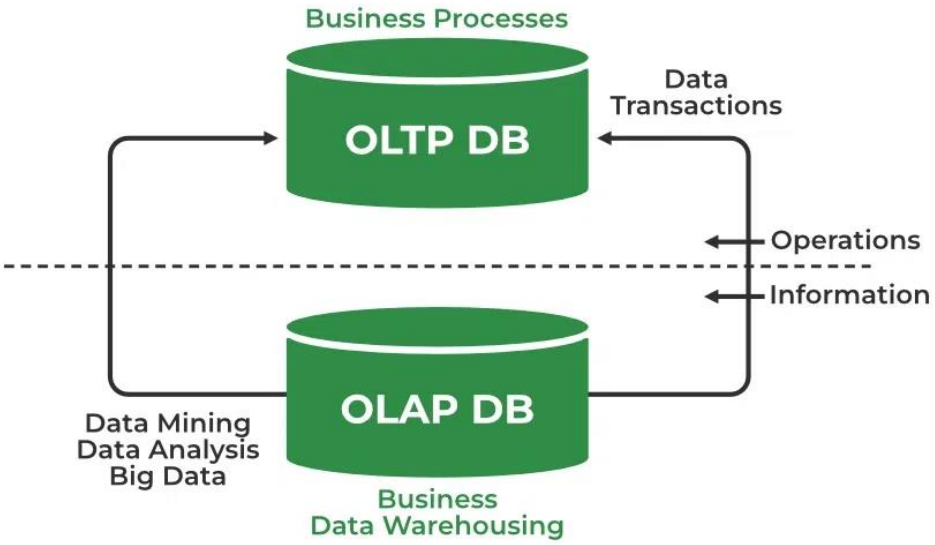
- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
- Operations:
  - Inserted when generated
  - Occasional update/delete
- Requirements
  - Low latency
  - High availability
- Typically row-major

Row

```
INSERT INTO RideTable(RideID, Username, DriverID, City, Month, Price)
VALUES ('10', 'memelord', '3932839', 'Stanford', 'July', '20.4');
```

OnLine Analytical Processing (OLAP)

- How to get aggregated information from a large amount of data?
  - e.g. what's the average ride price last month for riders at Stanford?
- Operations:
  - Mostly SELECT



Source

OnLine Analytical Processing

- Analytical queries: aggregated information from a large amount of data?
  - e.g. what's the average ride price last month for riders at Stanford?
- Operations:
  - Mostly SELECT
- Requirements:
  - Can handle complex queries on large volumes of data
  - Okay response time (seconds, minutes, even hours)

OnLine Analytical Processing

- Analytical queries: aggregated information from a large amount of data?
  - e.g. what's the average ride price last month for riders at Stanford?
- Operations:
  - Mostly SELECT
- Requirements:
  - Can handle complex queries on large volumes of data
  - Okay response time (seconds, minutes, even hours)
- Typically column-major

Column

```
SELECT AVG(Price)
FROM RideTable
WHERE City = 'Stanford' AND Month = 'July';
```

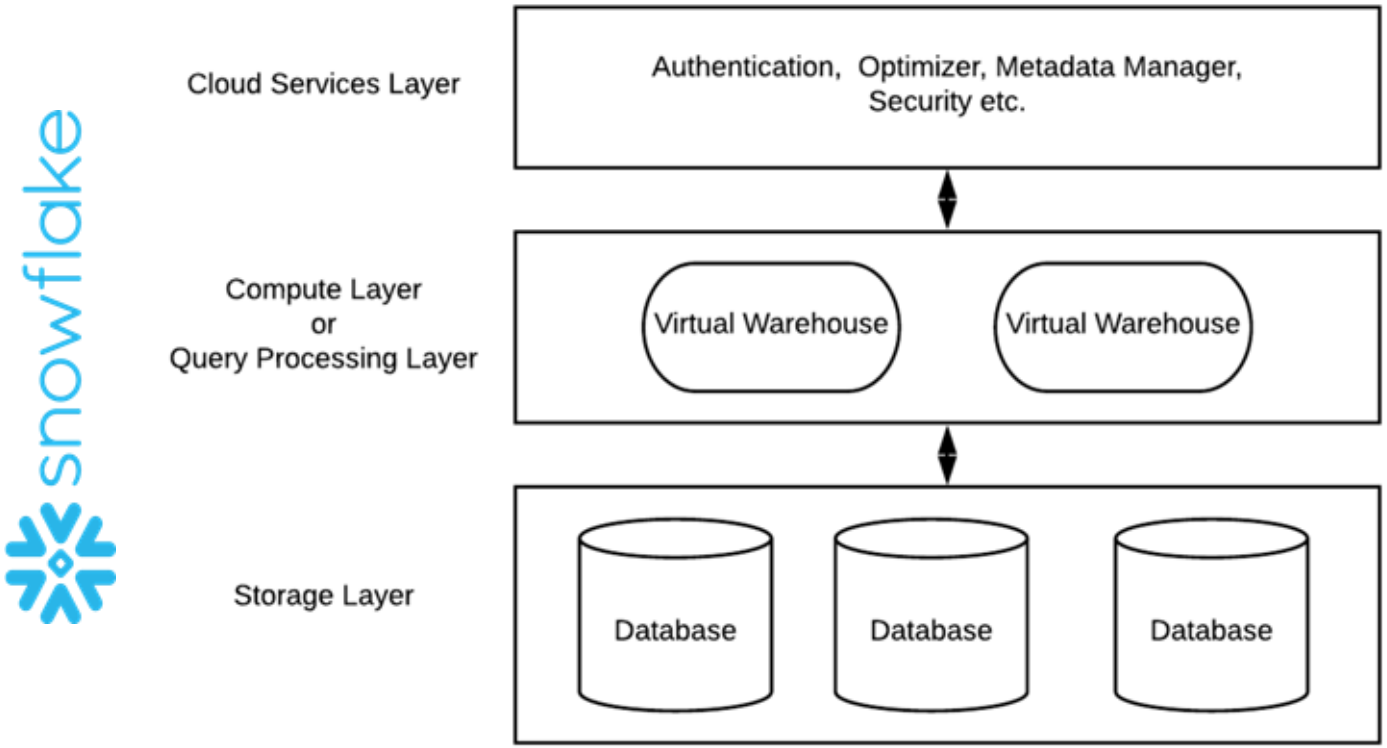
OLTP & OLAP are outdated terms

Decoupling storage & processing

- OLTP & OLAP: how data is stored is also how it's processed
  - Same data being stored in multiple databases
  - Each uses a different processing engine for different query types
  - **How data is stored dictates how it is processed**
- New paradigm: storage is decoupled from processing
  - Data can be stored in the same place (object storage like S3)
  - Storage is cheap, durable, and shared
  - A processing layer on top that can be optimized for different query types (SQL engines, spark, ML pipelines, etc)



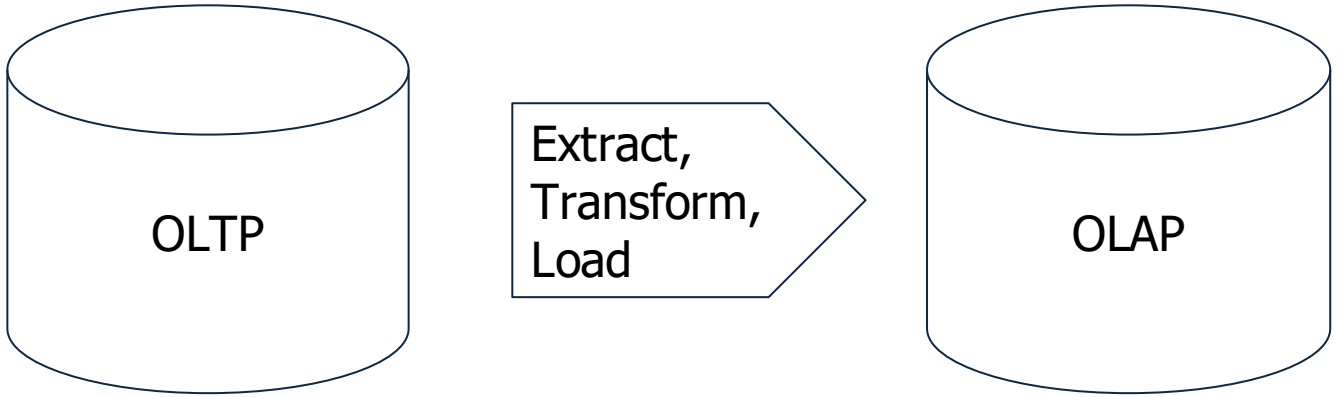
Decoupling storage & processing



Source

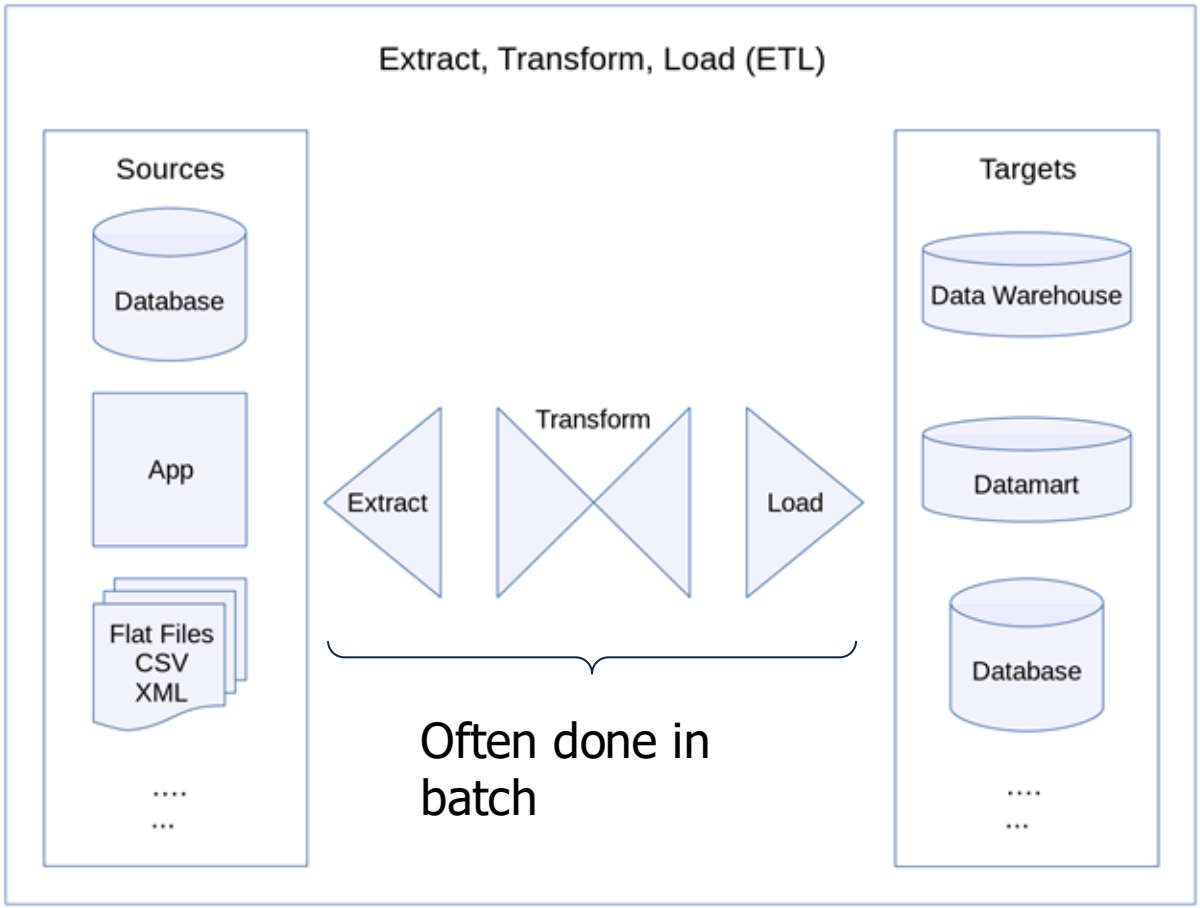

ETL
LIGHTYEAR
ETL EVERYWHERE

ETL (Extract, Transform, Load)



Transform: the meaty part

- cleaning, validating, transposing, deriving values, joining from multiple sources, deduplicating, splitting, aggregating, etc.



Source

ETL Adapting

Traditional Model

- OLTP and OLAP used different databases
- Data had to be copied and transformed
- ETL existed to move data from transactional systems into analytical systems
- Example flow:
  - OLTP DB → ETL → OLAP DB
- ETL was required because storage and processing were bound together.

Modern Model (Decoupled Storage & Compute)

- Data is stored once (data lake / object storage)
- Multiple processing engines query the same data
- ETL shifts from:
  - *Moving data between databases*
  - To *preparing and structuring data* for different workloads
- Example flow:
  - OLTP DB → ETL → Data Lake → Multiple Engines

THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

What to work on this week

What To Work On This Week

- Homework #3 (Hands-on Assignment)
- Reading: Chapters 1 of Data Engineering Fundamentals on O'Reilly
  - Remember to check the syllabus for assigned readings